

Apple SPMI Specification

Revision 1.3.2

04/19/2019

Abstract

This document contains the Apple SPMI PMU Specification.

Apple SPMI Specification

Revision 1.3.2

04/19/2019

Author

Inder Sodhi inder@apple.com

Copyright © 2019 Apple Inc. All rights reserved.

This material contains certain trade secrets and confidential and proprietary information of Apple Inc. Use, reproduction, disclosure, or distribution by any means are prohibited, except pursuant to a written license from Apple. Use of copyright notice is precautionary and does not imply publication or disclosure. Apple and the Apple logo are trademarks of Apple Inc., registered in the U.S. and other countries.



Contents

Styles	7
1. Apple SPMI Spec	8
1.1. Revision History	8
1.2. Overview	9
1.2.1. Goals	9
1.2.2. High Level Requirements	10
1.3. SPMI Protocol	11
1.3.1. MIPI Timing Spec	11
1.3.2. Timing Parameters	11
1.4. Commands	13
1.4.1. Command Format	13
1.5. Interrupts	13
1.5.1. Device Interrupts	14
1.5.2. Virtual Wires on SPMI	17
1.5.3. SPS (System Power State) ¹	20
1.5.3.1. Rules for the Power State Flow Function	20
1.5.4. DVC_Group ²	21
1.5.4.1. Rules for the DVC_Group Flow Function	21
1.5.5. DVC (Dynamic Voltage Change) ³	22
1.5.5.1. Rules for the DVC Flow Function	23
1.5.6. Interrupt handling	24
1.5.6.1. Edge Sensitive	25
1.5.6.2. Level Sensitive	25
1.5.6.3. Interrupt Priority	26
1.6. Short Addressing Modes	27
1.7. Register0 Write	28
1.8. Reset/Sleep/Shutdown/Wakeup	28
1.9. SGPIO	28
1.10. Registers	34
1.10.1. SPMI Configuration	34
1.10.1.1. SPMI_OLAT_REG_ADDR	36
1.10.1.2. SPMI_CTRL	37
1.10.1.3. SPMI_REVISION	37
1.10.1.4. MASTER_ID	37
1.10.1.5. READ_LATENCY	37
1.10.1.6. INTERRUPT_ADDR	38
1.10.1.7. INTERRUPT_OFFSET	38
1.10.1.8. DEVICE_CAPABILITY_TYPE	38
1.10.1.9. DEVICE_CAPABILITY_CMD	39
1.10.1.10. INTERRUPT_VW_ADDR	40
1.10.1.11. MASTER_PMU_ID	40
1.10.1.12. SGPIO_SLAVE_ID	40

¹System PMU Specific

²DVC group capable PMU Specific

³DVC capable PMU Specific

1.10.1.13. GROUP_SLAVE_ID	40
1.10.1.14. EXT_{READ,WRITE}_ADDRESS	40
1.10.1.15. ADDRPOINTER{1,2,3}_{LOW,HIGH}	41
1.10.1.16. ARB_MON_TIMEOUT	41
1.10.1.17. CMD_SEQ_MON_TIMEOUT	41
1.10.1.18. ARB_REQ_HOLD_CTRL	41
1.10.1.19. ERROR_CTRL	42
1.10.1.20. IRQ_ENABLE	42
1.10.1.21. EVT_PRIO_REG_EVT{0 – 255}	42
1.10.2. Power State and DVC Registers ⁴	42
1.10.3. DVC ⁵	46
1.10.4. DVC_GROUP Configuration ⁶	47
1.11. Examples of SPMI transactions ⁷	49
1.11.1. Power State Transition	49
1.11.2. Single Rail DVC Transition ⁸	50
1.11.3. DVC Group Transition ⁹	51
1.11.4. DVC Early_Done ¹⁰	54
1.11.5. Backlight Control ¹¹	57
1.12. ERROR Handling	58
1.12.1. RCS – Master Write	59
1.12.2. RCS Transaction Start Error	59
1.12.3. SPMI Protocol Error	60
1.13. Additional Notes	61
1.13.1. Power State Transition [Examples]	63
1.14. Non-compliance with MIPI Spec	66
1.15. Clarifications from earlier Specs	66
1.15.1. Slave Hold Behavior	66
1.15.2. Command and Arbitration Timeout values	66
1.15.2.1. ADC data read back enhancement	67
1.15.3. Slave to Slave transactions	67
1.15.4. Multiple Instantiations of same silicon device	68

⁴System PMU Specific⁵DVC capable PMU Specific⁶DVC group capable PMU Specific.⁷System PMU Specific⁸DVC capable PMU Specific⁹DVC group capable PMU Specific¹⁰DVC capable PMU Specific¹¹Backlight capable device Specific

List of Figures

1.1. Single Master/Slave	10
1.2. Multiple Slaves	11
1.3. Trace model parameters	12
1.4. SPMI and Throttle triggers - Timing diagram	17
1.5. SPMI Rev 1.2: Device Interrupts	19
1.6. Example of a 2 PMU system - Buck/PD mapping	23
1.7. Example of multi-slave DVC mapping	24
1.8. Example of Device Interrupt grouping	26
1.9. SGPIO interface - system	29
1.10. SGPIO - Slave Config	30
1.11. SGPIO - Power Sequencer	31
1.12. System Power State transition	49
1.13. Buck DVC transition	50
1.14. Buck group transition (Time Domain)	52
1.15. Buck group transition (Voltage Domain)	53
1.16. Buck group transition (different target)	54
1.17. Buck group transition (same target)	54
1.18. Buck DVC (Allow_off=1)	55
1.19. Buck DVC (Allow_off=0)	55
1.20. Buck DVC (Allow_off=1, Early_Done=0)	56
1.21. Buck DVC (Allow_off=1, Early_Done=1)	56
1.22. RCS initiated Master Write transaction	59
1.23. RCS, Master Write – Start Error	60
1.24. RCS, Master Write – Protocol Error	61
1.25. ERRRL	62
1.26. ERRRL with Delay	62
1.27. Power State transition (Awake → Sleep1)	64
1.28. Power State transition (New Target after Threshold)	65
1.29. Power State transition (Awake → Sleep1 w/ Abort)	65

List of Tables

1.1. Revision History	8
1.2. SPMI Timing Parameters	11
1.3. SCLK/SDATA Timing Parameters	12
1.4. Master Writes	13
1.5. Interrupts	14
1.6. Interrupts [191:0]	15
1.7. Device Specific Interrupts (Example 1)	15
1.8. Device Specific Interrupts (Example 2)	15
1.9. Top Level Device Specific Interrupts	16
1.10. Power State Interrupts	20
1.11. DVC Group Done Interrupts	21
1.12. Buck DVC Done Interrupts	22
1.13. Short Addressing Mode: Pointers	27
1.14. Register Map	34
1.15. SPMI Configuration	34
1.16. Device capability_type bits	38
1.17. Device capability_cmd bits	39
1.18. Power State	43
1.19. Power State Registers	45
1.20. DVC VSEL Registers	46
1.21. DVC Group Configuration	47
1.22. DVC Group Target Registers	48
1.23. WLED - PSPI	57
1.24. BLC - PSPI	57
1.25. WLED & BLC - SPMI	57



Styles

SEG has adopted Section 13.1 of the IEEE Standards Style Manual, which dictates the use of the words *shall*, *should*, *may*, and *can* in the development of documentation, as follows:

- The word *shall* is used to indicate mandatory requirements strictly to be followed and from which no deviation is permitted (*shall* equals *is required to*).
 - The use of the word *must* is deprecated and shall not be used when stating mandatory requirements; *must* is only used to describe unavoidable situations.
 - The use of the word *will* is deprecated and shall not be used when stating mandatory requirements; *will* is only used in statements of fact.
- The word *should* is used to indicate that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain course of action is deprecated but not prohibited (*should* equals *is recommended that*).
- The word *may* is used to indicate a course of action permissible within the limits of the specification (*may* equals *is permitted*).
- The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).
- The words *that* and *which* are commonly misused; they are not interchangeable. *That* is best reserved in essential (or restrictive) clauses, *which* is appropriate in nonessential (or nonrestrictive), parenthetical clauses. Simply state, if a comma can be inserted before the word *that* or *which* , the word should be *which* . If a comma would not be used, the word to use is *that* .
- All sections are normative, unless they are explicitly indicated to be informative.
- Numbers are decimal unless otherwise indicated. A prefix of 0x indicates a hexadecimal number, while a prefix of 0b indicates a binary number.



Chapter 1. Apple SPMI Spec

1.1 Revision History

Table 1.1. Revision History

Revision	Date	Owner	Change Summary
1.3.2	4/8/2019	Inder Sodhi	Updated the registers for SGPIO (Host command)
1.3.1	3/18/2019	Inder Sodhi	- Added back the ERROR_EVENT and ERROR_MASK registers - Updated the TimeOut defaults in the registers
1.3	2/14/2019	Inder Sodhi	- Updated SGPIO and vGPIO specification - Added details of SGPIO_CMD register interface
1.2.3	1/9/2019	Inder Sodhi	- Clarified Slave-to-Slave requirements - Added option to relax CLK-->DATA timing spec
1.2.2	11/27/2018	Inder Sodhi	- Added SPMI event for Force_DFU - Fixed Display capability section
1.2.1	10/17/2018	Inder Sodhi	- Incorporated feedback from internal reviews. - Added details on SGPIO interface.
1.2	6/1/2018	Inder Sodhi	- SPMI configuration update - Added Device_Cap_type & Device_Cap_Cmd - Added VW (Virtual Wire support) - Added P_SPMI (PMU SPMI Master support) - Moved some device config registers to SPMI address space - Event register consolidation - Device Specific Interrupt Mapping Registers - Moved some events to different IDs - In-band Event Set/Clear support - Master_Write_Addr=1 for Virtual Wire event messaging - ADC Accumulators, Registers - ADC result registers for automatic scheduler are in continuous address space

Revision	Date	Owner	Change Summary
			◦ Reduce # SPMI transactions for data collection, allow HW-assist in SoC for data gathering
1.1	10/12/2017	Inder Sodhi	• Made the device interrupts more generic. • Modified device capability register space ◦ added slave to slave communication ◦ added support for 0-latency registers 0-0xFF
1.062	07/10/2017	Inder Sodhi	• Changed the WLED command syntax to be LSB first then MSB
1.061	05/29/2017	Inder Sodhi	• Added details on multi-slave support
1.06	05/04/2017	Inder Sodhi	• Clarified System PMU vs. Companion PMU requirements ◦ Athena/Apollo development support
1.05	01/18/2017	Inder Sodhi	• Updated description for “Device Capability” register. • Clarified the slave_hold and timeout behavior details.

1.2 Overview

System Power Management Interface (SPMI) is a two-wire serial interface that connects the integrated power controller of a System-on-Chip (SoC) processor with one or more Power Management Integrated Circuits (PMIC) voltage regulation systems and other devices that participate in the power architecture in the system.

This document proposes a method of communication and functions between Master (aka Host) and Slave devices (aka PMIC, PMU) using SPMI transactions, with the goal of eliminating the need for additional external wires for these devices (for ex: interrupt pins etc.).

SPMI is a high speed interconnect that enables systems to communicate, exchange Telemetry and Event information without additional pins and dynamically adjust the supply voltages of the voltage domains inside the SoC and other devices in the system.

Apple follows the “*MIPI Alliance Specification for System Power Management Interface (SPMI)*”, Version 2.0 – 3/14/2012, approved by the MIPI Board on 8/28/2012.

1.2.1 Goals

The goals of the SPMI interface are:

- Remove special commands and use industry standard commands for all communication.
- Add flexibility that allows multiple voltage rails control simultaneously.
- Improved Telemetry and a low latency read path of captured data for advanced thermal and power management algorithms.

- Provide access to all memory mapped device internal registers and scratchpad SRAM
- In-band handling of interrupts with configuration to reduce software overhead of interrupt processing.

Goal of this document

This document is meant to identify the Apple Specific implementation of a SPMI slave specification. It is written as a delta document with respect to the spec, so whenever something is not specified, the reader should refer to the spec for details.

The target audience for this document is designers and micro-architects who are building SPMI “slave” devices that will be used in Apple systems.

1.2.2 High Level Requirements

The Slave SPMI implementation shall meet these requirements

1. A Request Capable Slave with Slave Arbitration process support.
2. Supports the High Speed Class interface

Figure 1.1 shows a typical configuration (one Master and one Slave).

Figure 1.1. Single Master/Slave

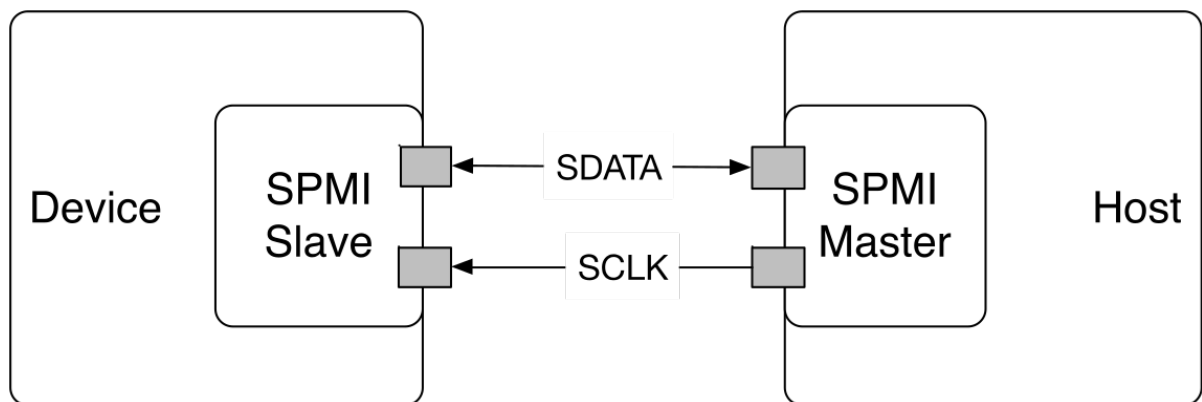
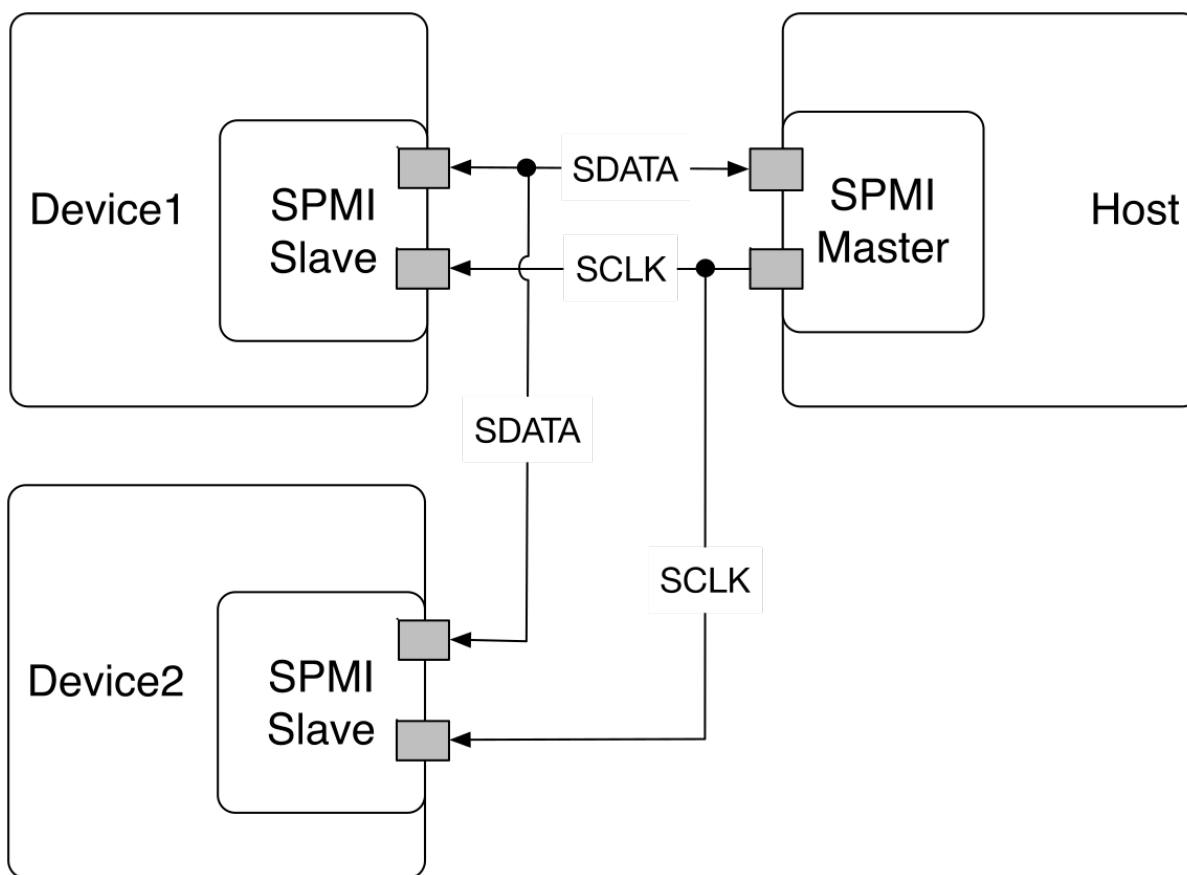


Figure 1.2 shows a configuration that has multiple Slaves in a system

Figure 1.2. Multiple Slaves



1.3 SPMI Protocol

1.3.1 MIPI Timing Spec

Supports the High Speed class interface (24 MHz SCLK) as specified in section 5.4.1 in the MIPI SPMI specification.

Table 1.2. SPMI Timing Parameters

Description	Min	Max	Unit
SPMI Clock Frequency	6	24	MHz
SPMI Clock Period	41.67		ns
SPMI Clock Duty Cycle	45	55	%

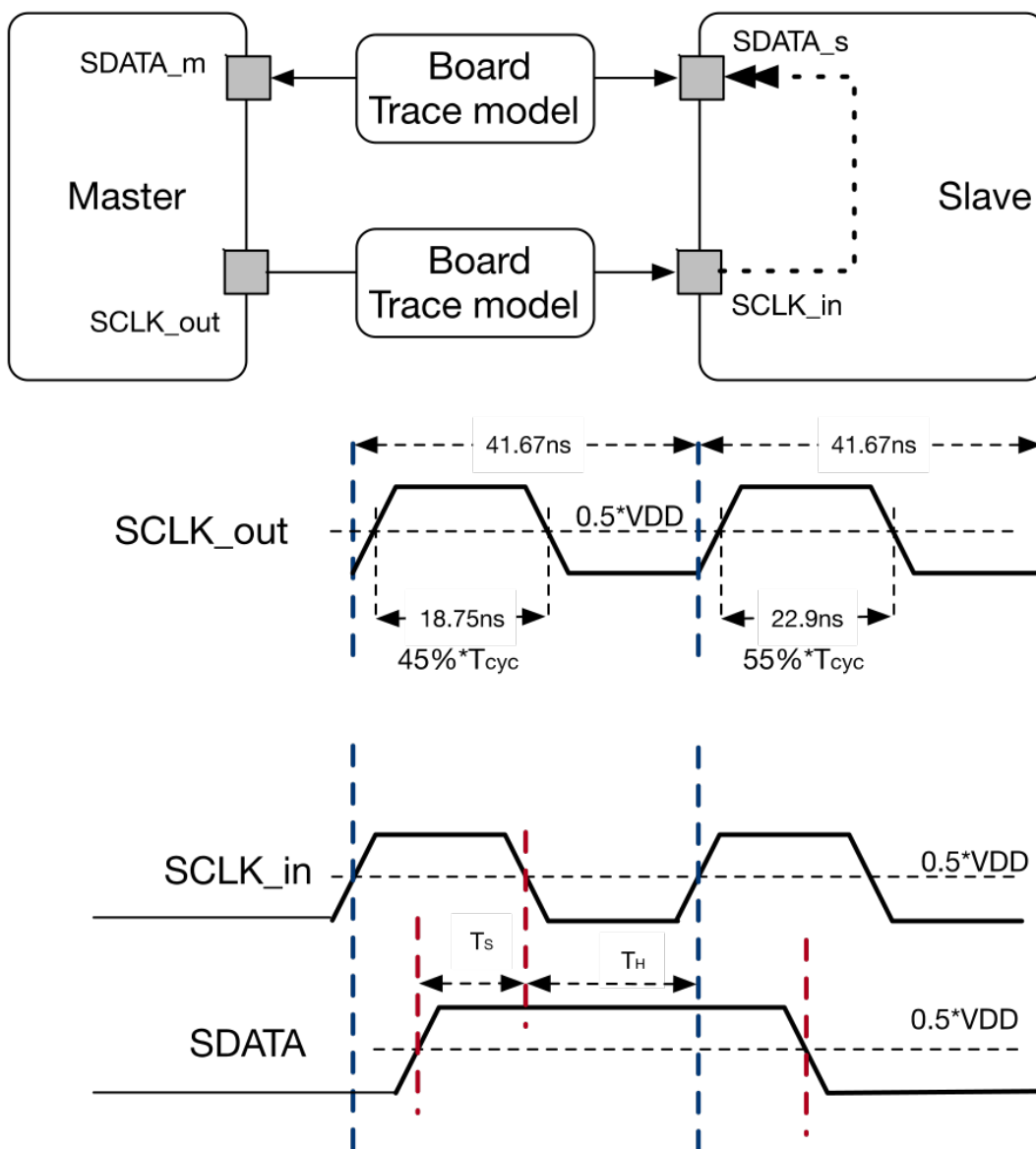
A SPMI RCS can support either 1.2V or 1.8V signaling or both.

The device voltage level support needed depends on the device ERS.

1.3.2 Timing Parameters

For the SI analysis for the SPMI bus, the following model is used for the Master/Slave drive strength and timing analysis.

Figure 1.3. Trace model parameters



The Master (Host) drives on the rising edge of SCLK and the Slave (Device) samples on the falling edge of SCLK. However, for the slave to generate transactions on the bus, there is a half cycle path; i.e.; Slave samples the clock (SCLK_in) and then drives its data out (SDATA_out)

These are the electrical parameter (ranges) for 2 different kinds of systems using SPMI.

Table 1.3. SCLK/SDATA Timing Parameters

Parameter	System 1: w/ DeSense Cap	System 2: w/ DeSense Cap	System 3: w/o DeSense Cap	Unit
Host Setup Time	5	5	3	nS
Host Hold Time	2	2	2	nS
Host Clock-to-Data	7	11	4	nS

Parameter	System 1: w/ DeSense Cap	System 2: w/ DeSense Cap	System 3: w/o DeSense Cap	Unit
Drive Strength	8	8	4	mA
C_Load range	40 → 50	40 → 50	20 → 30	pF

Some systems need further relaxation on the Clock → data path, the spec for which can be increased from 7ns to 11ns. This will make the data capture setup path inside the slave tighter. It should also make the data capture path inside the master tighter too (as slave driver will also get the relaxed 11ns requirement).

The numbers above are estimates, for additional requirements on the signaling levels, drive strength, slew rate etc.; please refer to the relevant system specification documents.

1.4 Commands

No changes, the commands supported are based on spec.

1.4.1 Command Format

No change from the Spec.

1.5 Interrupts

Interrupts are signaled from the Slave to the Master using MASTER_WRITE command on SPMI.

To do this, the slave will generate a write to a special address (aka Interrupt_Address) which is a value in the slave that is configured by the Host during Boot.

The Interrupt_Address is a 8b value that is configured in the device by Host S/W or OTP bits in the device, the default value is 0x00.

This special address in the Master, which is an indication to it that a RCS (Request Capable Slave) wants to get its attention for interrupt handling. The data size is 1 byte only.

As an example, if the slave wants to send an interrupt (0xB2) to the master, the transaction will look like this:

Table 1.4. Master Writes

	Master Write Command Frame	Data Frame with Register Address	Data Frame with Write Data
Request	'00, <Master_ID[1:0]>, 0x16	<Interrupt_Address>	0xB2
Example:	'0000_0001_0110	'0000_0000	'1011_0010

If the Master Write transaction from the Slave is “NACK” by the Host, it’s an Error condition. The Slave will have a register that configures the number of “retires” that the slave will issue on receiving a NACK from the Host.

Rules are:

- Slave retires the transaction until the retry counter expires (default = 3).

- If any of these transactions gets a successful ACK, the counter is reset.
- Depending on retry counter configuration in the slave, when there are X consecutive NACKs from the Master, the slave goes into Error handling which can be SW or OTP configured:
- Error handling for System PMU:
 - If set, the slave will log the error as a fault and do a RESET sequence, which will clear the error counter and the Master will read the fault log to do error handling.
 - If cleared, the slave will stop issuing Master Write transactions by clearing IRQH enable and waits until Master re-enables IRQH enable capability.
- Error handling for non-System PMU:
 - If set, the slave will stop issuing Master Write transactions by clearing IRQH enable and waits until Master re-enables IRQH enable capability.
 - If cleared, the slave will continue issuing the transaction as allowed by SPMI protocol.
- Disabling the IRQH enable implies that slave is not able to send Master Write transactions on the SPMI bus, it has no effect on ability of slave to service read/write transactions from the master or on the ability to capture and latch events as programmed.

Apple SPMI Master supports 256 interrupts and the table below shows how they are partitioned.

Table 1.5. Interrupts

Size	Bits	Type	Description
192	[191:0]	DEVICE_SPECIFIC	Generic interrupts based on devices in the system
8	[199:192]	POWER_STATE_DONE ¹	System Power State transition completed
8	[207:200]	DVC_GROUP_DONE ²	DVC complete on Group[x]
16	[223:208]	BUCK_DVC_DONE ³	Voltage change transition (DVC) completed on a particular BUCK
16	[239:224]	LDO_DVC_DONE ³	Voltage change transition (DVC) completed on a particular LDO
16	[255:240]	RSVD	Reserved

¹These are relevant for System PMU only.

²These are relevant for DVC Group capable PMU only.

³These are relevant for DVC capable PMU only.

The next sections go into details of each of these interrupt types.

1.5.1 Device Interrupts

There are 192 generic interrupts that can be sent from the slave to the master. These interrupts should be configured so that each device will not allocate more than 64 interrupts.

To allow flexibility on adjusting the device specific range, each slave will support an 8b OFFSET register that will be used to offset the device specific registers.

A generic example of N devices with M_x sized interrupt vectors in shown in the table below.

Table 1.6. Interrupts [191:0]

Size	Bits	Type	Description
M_0	$[M_0-1:0]$	Device0	Offset = 0
M_1	$[M_1-1: M_0]$	Device1	Offset = M_0
...	[...]	...	...
M_n	$[191: M_{n-1}]$	DeviceN	Offset = M_{n-1}

The total size of the “device specific” interrupt range for each Master is 192 bits,

i.e.; $M_0 + M_1 + \dots + M_n \leq 192$

An example of a system with 3 devices is shown below. Each device supports device specific interrupts in the [63:0] range.

Table 1.7. Device Specific Interrupts (Example 1)

Size	Bits	Type	Description
64	[63:0]	Device0	Single device generic interrupts (offset = 0) Range = [63:0]
64	[127:64]	Device1	Single device generic interrupts (offset = 64) Range = 64 + [63:0]
64	[191:128]	Device2	Single device generic interrupts (offset = 128) Range = 128 + [63:0]
64	[255:192]	RESERVED for System PMU	System PMU specific interrupts (offset = 192) Range = 192 + [63:0]

An example of a system with 3 devices is shown below. One of the devices supports 64 interrupts and the other 2 only support 32 interrupts.

Table 1.8. Device Specific Interrupts (Example 2)

Size	Bits	Type	Description
64	[63:0]	Device0	Single device generic interrupts (offset = 0) Range = [63:0]
32	[95:64]	Device1	Single device generic interrupts (offset = 64) Range = 64 + [31:0]
32	[127:96]	Device2	Single device generic interrupts (offset = 96) Range = 96 + [31:0]
64	[191:128]	unused	
64	[255:192]	RESERVED for System PMU	System PMU specific interrupts (offset = 192) Range = 192 + [63:0]

This architecture requires a slave device to have the offset capability register so that devices can be built using a generic interrupt range. Since the number and type of other devices are based on a system architecture, a device that has been built as a generic [63:0] interrupt range device, will use the offset register so that the host can distinguish interrupts from different slave devices.

This is the Top Level Device Interrupt structure for PMU devices.

Table 1.9. Top Level Device Specific Interrupts

	0	1	2	3	4	5	6	7
00--07	SYSCTL_GROUP	MFSM_GROUP	RTC	SPMI_GROUP	RESET_WARN	BTN2_DBL	FORCE_DFU	RSVD
08--0F	BTN_1	BTN_2	BTN_3	BTN_4	CHG_GROUP_A	CHG_GROUP_B	WLED_GROUP_A	WLED_GROUP_B
10--17	GPIO_1	GPIO_2	GPIO_3	GPIO_4	GPIO_5	GPIO_6	GPIO_7	GPIO_8
18--1F	GPIO_9	GPIO_10	GPIO_11	GPIO_12	GPIO_13	GPIO_14	GPIO_15	GPIO_16
20--27	GPIO_17	GPIO_18	GPIO_19	GPIO_20	GPIO_21	GPIO_22	GPIO_23	GPIO_GROUP
28--2F	THRT_1	THRT_2	THRT_3	THRT_4	THRT_5	THRT_6	THRT_7	THRT_8
30--37	THRT_9	THRT_10	THRT_11	THRT_12	THRT_13	THRT_14	THRT_15	THRT_16
38--3F	ADC_EOC_1	ADC_EOC_2	ADC_SCAN1_EOC	ADC_SCAN2_EOC	GPADC_GROUP	THRT_GROUP	RSVD	RSVD

Here is a short description of the various device events. For additional details, refer to the device ERS/Spec.

- SYSCTL_GROUP: Highest priority system events, like high temp or button reset sequence.
- MFSM_GROUP: FSM errors, including invalid state transitions.
- RTC: Real Time Clock alarm event.
- SPMI_GROUP: Internal or bus errors on the SPMI interface.
- RESET_WARN: Warning signal to Host that a Reset event is coming.
- BTN2_DBL: Double click event on Button2.
- FORCE_DFU: Force Device Firmware Update event.
- BTN_x: Button X event.
- CHG_GROUP_x: Charger related event.
- WLED_GROUP_x: Backlight related event.
- GPIO_x: GPIO X event.
- THRT_x: Throttle X event.

- ADC_EOC_x: End of Conversion for ADC channel x.
- GPADC_GROUP: Grouped events for GPADC.
- THRT_GROUP: Grouped events for Throttle logic.

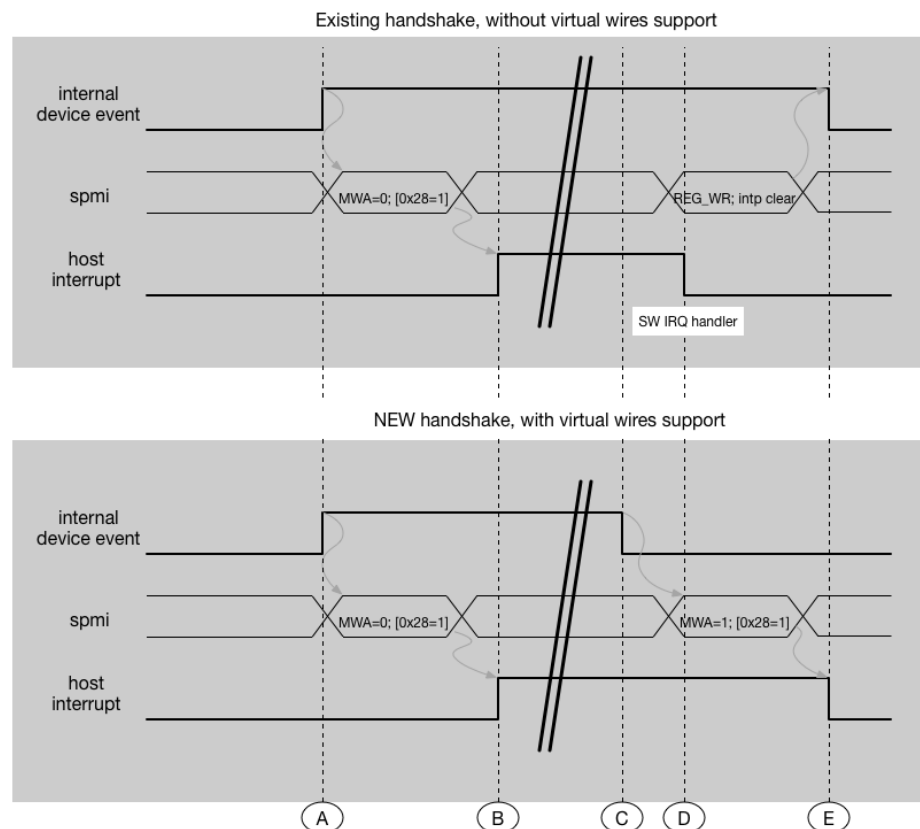
1.5.2 Virtual Wires on SPMI

In addition, devices will add the ability to support virtual wires (events) in-band using SPMI.

The host SW will configure one or more of the 16 device interrupts from the SPMI device to set/clear any of the internal throttle trigger signals in the SoC. These SPMI events are in addition to the external trigger pins.

The logic in Host SPMI master will monitor (set and clear) the throttle triggers based on in-band events from the device. An example is shown in the figure below: The first portion is the event getting triggered using the wire only. The second portion shows that SPMI engine is configured to send interrupt to set/clear one of the CPU trigger pins.

Figure 1.4. SPMI and Throttle triggers - Timing diagram



The figure above shows both the existing IRQ handling and the new architecture with virtual wires

In the existing handshake:

A: internal event (or GPIO) on device asserts which results in SPMI transaction to Host. Device generates a Master_Write_Addr=0 event with THRTL1 event (0x28) set

B: Host gets the event on the SPMI bus, generates an IRQ to Host SW

D: SW IRQ handler services the interrupt, generates a Register_Write command on SPMI to clear the event in the device.

E: Event is cleared in device.

In the new handshake:

A: internal event (or GPIO) on device asserts which results in SPMI transaction to Host. Device generates a Master_Write_Addr=0 event with THRTL1 event (0x28) set

B: Host gets the event on the SPMI bus, generates an IRQ to Host SW

C: Event in device de-asserts.

D: Event de-assertion results in SPMI transaction to Host. Device generates a Master_Write_Addr=1 event with THRTL1 event (0x28) set, to clear the event.

E: Event is cleared in host.

Based on the SPMI interrupt spec, only 0x28-0x37 events (16 interrupts) are available in the device for triggering Throttle events (aka virtual wires or in-band throttle signals). As shown in the table below:

Figure 1.5. SPMI Rev 1.2: Device Interrupts

RCS: Master_Write_Address=0x0

	0	1	2	3	4	5	6	7
00--07	SYSCTL_GROUP	MFSM_GROUP	RTC	SPMI_GROUP	RESET_WARN	BTN2_DBL	RSVD	RSVD
08--0F	BTN_1	BTN_2	BTN_3	BTN_4	CHG_GROUP_A	CHG_GROUP_B	WLED_GROUP_A	WLED_GROUP_B
10--17	GPIO_1	GPIO_2	GPIO_3	GPIO_4	GPIO_5	GPIO_6	GPIO_7	GPIO_8
18--1F	GPIO_9	GPIO_10	GPIO_11	GPIO_12	GPIO_13	GPIO_14	GPIO_15	GPIO_16
20--27	GPIO_17	GPIO_18	GPIO_19	GPIO_20	GPIO_21	GPIO_22	GPIO_23	GPIO_GROUP
28--2F	THRT_1	THRT_2	THRT_3	THRT_4	THRT_5	THRT_6	THRT_7	THRT_8
30--37	THRT_9	THRT_10	THRT_11	THRT_12	THRT_13	THRT_14	THRT_15	THRT_16
38--3F	ADC_EOC_1	ADC_EOC_2	ADC_SCAN1_EOC	ADC_SCAN2_EOC	GPADC_GROUP	THRT_GROUP	RSVD	RSVD

RCS: Master_Write_Address=0x1 (w/o VIRTUAL_GPIO)

	0	1	2	3	4	5	6	7
00--07	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD
08--0F	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD
10--17	GPIO_1_CLR	GPIO_2_CLR	GPIO_3_CLR	GPIO_4_CLR	GPIO_5_CLR	GPIO_6_CLR	GPIO_7_CLR	GPIO_8_CLR
18--1F	GPIO_9_CLR	GPIO_10_CLR	GPIO_11_CLR	GPIO_12_CLR	GPIO_13_CLR	GPIO_14_CLR	GPIO_15_CLR	GPIO_16_CLR
20--27	GPIO_17_CLR	GPIO_18_CLR	GPIO_19_CLR	GPIO_20_CLR	GPIO_21_CLR	GPIO_22_CLR	GPIO_23_CLR	RSVD
28--2F	THRT_1_CLR	THRT_2_CLR	THRT_3_CLR	THRT_4_CLR	THRT_5_CLR	THRT_6_CLR	THRT_7_CLR	THRT_8_CLR
30--37	THRT_9_CLR	THRT_10_CLR	THRT_11_CLR	THRT_12_CLR	THRT_13_CLR	THRT_14_CLR	THRT_15_CLR	THRT_16_CLR
38--3F	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD

RCS: Master_Write_Address=0x1 (w VIRTUAL_GPIO)

	0	1	2	3	4	5	6	7
00--07	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD
08--0F	THRT_1_SET	THRT_2_SET	THRT_3_SET	THRT_4_SET	THRT_5_SET	THRT_6_SET	THRT_7_SET	THRT_8_SET
10--17	THRT_9_SET	THRT_10_SET	THRT_11_SET	THRT_12_SET	THRT_13_SET	THRT_14_SET	THRT_15_SET	THRT_16_SET
18--1F	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD
20--27	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD
28--2F	THRT_1_CLR	THRT_2_CLR	THRT_3_CLR	THRT_4_CLR	THRT_5_CLR	THRT_6_CLR	THRT_7_CLR	THRT_8_CLR
30--37	THRT_9_CLR	THRT_10_CLR	THRT_11_CLR	THRT_12_CLR	THRT_13_CLR	THRT_14_CLR	THRT_15_CLR	THRT_16_CLR
38--3F	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD	RSVD

This scheme will be used only for the current/imaxT limits (~100us) averages, not voltage comparator related.

Two schemes of usage/mapping are shown above.

- In the first scheme, the same ID is used and MWA=0 for set & MWA=1 for clear
 - As an example: for event THRTL_1, the PMU will generate the following interrupts:
 - MWA=0, ID=0x28 - throttle event set
 - MWA=1, ID=0x28 - throttle event clear
- In the second scheme, only MWA=1 is used, but a different ID is used for set & clear.
 - As an example: for event THRTL_1, the PMU will generate the following interrupts:
 - MWA=1, ID=0x08 - throttle event set
 - MWA=1, ID=0x28 - throttle event clear

1.5.3 SPS (System Power State)¹

System Power State is defined as a power state of the system/SoC that corresponds to the voltage levels of various Bucks, as configured by OTP of Host software.

The Master will generate a register write to ask the Slave/PMIC to transition the system to a particular state and when the system gets to the particular state, the slave will generate an interrupt to the Host.

Table 1.10. Power State Interrupts

Size	Bits	Type	Bits
8	[199:192]	Power_StateX_Done	Transition completed to Power_State[X].

These interrupt range structure covers 8 system power states that the Host will configure in the slave based on system design. If a particular system doesn't define or use a power state, the interrupt bit is reserved.

1.5.3.1 Rules for the Power State Flow Function

1. The Host will configure the desired state of each of the PD (Power Delivery block i.e.; Buck or LDO) for each of the power states supported in the system, including the power delay sequencer as needed for rail power up/down timing.
2. The slave will generate the "Done" interrupt as an indication to the Master that it is ready to accept a new SPS transition command. This can be configured in the PMU as needed.
3. The slave can expect that only "VALID" power state transitions will be generated from the Master, based on the slave power sequence FSM.

In the case that an error happens during the Power_State transition request.

Examples of these errors are

1. Transition request to non-existing arcs in the power state FSM.
2. Writing an invalid bit or fields in the target register.
3. Initiating a transition when the Power state transition is not allowed (Force_State register bit is set).

If any of these error conditions occur, the slave is expected to

1. Log the Error in appropriate status registers.
2. Generate an Error/Fault interrupt to the master as shown in the earlier (Device specific) tables.

A power state is defined in the slave as a configuration of enable/disables of the various PD blocks in that device.

¹System PMU Specific

An example of a system configuration is as follows:

[Power_State0] – AWAKE

[Power_State1] – SLEEP

...

[Power_State7] – OFF

An Example of OFF → AWAKE transitions timing flows is shown in the later sections.

1.5.4 DVC_Group²

Using SPMI register writes, the Master has the ability to simultaneously signal DVC to multiple Bucks in a single transition.

The interrupt structure supports a maximum of 7 groups of DVC Bucks, which is a configuration (OTP) set during system Reset.

Table 1.11. DVC Group Done Interrupts

Size	Bits	Type	Description
1	[207]	DVC_Group_Done_Error	This indicates that the slave was not able to finish the DVC transition for a particular DVC_Group.
7	[206:200]	DVC_Group_Done	Each bit here will indicate from the slave to the master that DVC for a particular DVC_Group has been completed. This configuration of which Bucks/LDOs are part of which group is S/W generated. The Host will program the slave with a configuration of interrupt bit position mapping to DVC_Group.

A DVC_Group is a group of PD (Buck/LDO) that will do a DVC transition together. A group can have a maximum of 4 PD blocks in it. The details of the registers and examples of the transitions timing flows are shown in the later sections.

1.5.4.1 Rules for the DVC_Group Flow Function

1. For each of the interrupts above, they are signaled from slave to Master on voltage transition “Done” of **ALL** the Bucks in the group. Done is defined as a state that the PMU can accept a new command for that group.
 - a. For power up transitions (DVC up), these will mean waiting for the transition to complete
 - b. For power down transitions (DVC down); it can be configured to be after transition is completed or can be configured to be pre-empted “ready for next command”. This will be specified in the ERS/configured in OTP for a particular system.

²DVC group capable PMU Specific

2. For DVC up transitions, the slave will not generate the interrupt until the all the PD (Buck/LDO) target has reached the new voltage level. This includes any “settling time” that is configured for that particular PD.
3. The slave can expect that for a particular Group, the master will not issue a new DVC request until it sees the “DVC_Group_Done” interrupt for an outstanding DVC request.

Interrupt [207] is reserved for the slave to indicate to the Host that an error has happened during the DVC_Group transition request. Examples of these errors are

1. Slave received a new DVC_Group request from Host before it indicated that a pending DVC transition has finished.
2. A particular Buck/LDO in the group does not have DVC enabled.
3. The power state of the PMU disables the Buck/LDO and so DVC is not possible for that Buck.

If any of these error conditions occur, the slave is expected to

1. Log the Error in appropriate status registers.
2. Generate a “DVC_Group_Error” write to the master as shown in the earlier tables.

1.5.5 DVC (Dynamic Voltage Change)³

When the slave gets a DVC register write for a particular PD (Power Delivery) IP (i.e.; a Buck or LDO), on completion of the voltage transition, this is the interrupt that it will send the Host informing it that the voltage transition for that particular Buck/LDO has been completed.

Table 1.12. Buck DVC Done Interrupts

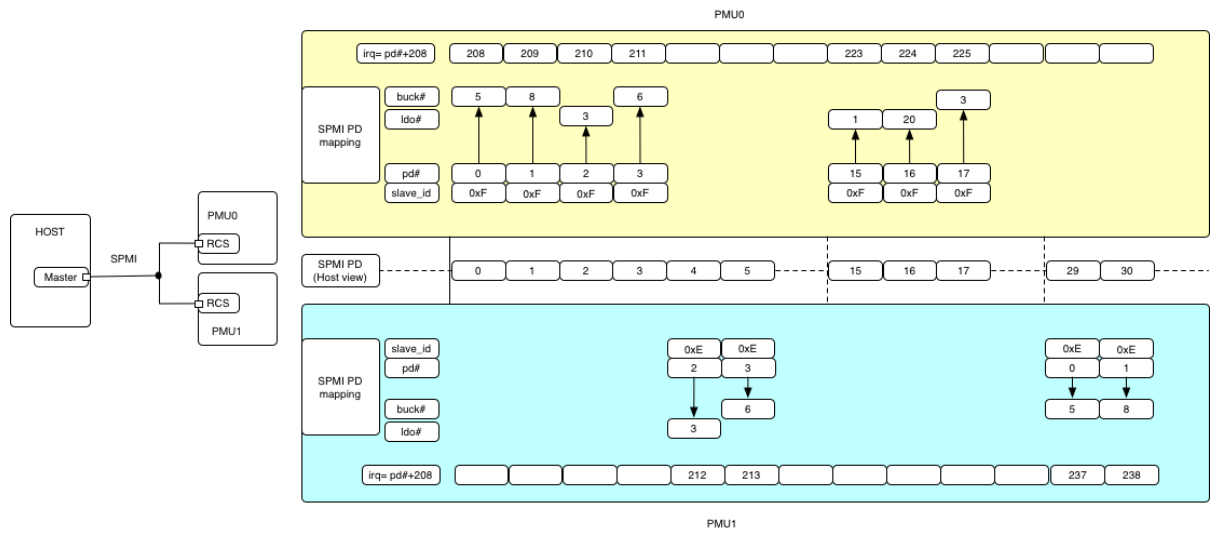
Size	Bits	Type	Description
1	[239]	PD_DVC_Done_Error	Slave is not able to complete the DVC command due to an error.
31	[238:208]	PD_DVC_Done_X	Each bit here will indicate from the slave to the master that DVC for a particular Buck has been completed. This configuration is S/W generated. The Host will program the slave with a mapping of Buck/LDO with “DVC_Done” bit position.

The architecture supports 31 DVC capable PD (Bucks/LDOs) in each slave. Those slave devices that don't support the respective Bucks, will keep these interrupts bits UNUSED.

Examples of the transitions timing flows are shown in the later sections. To illustrate the Buck-PD mapping, here is an example of how a 2 PMU system will get designed.

³DVC capable PMU Specific

Figure 1.6. Example of a 2 PMU system - Buck/PD mapping



For each of the slave devices (PMU 0/1, in the example above) the index of each PD is unique. The PD → Buck/LDO mapping is configurable (using OTP) as desired in the system.

- The index of the PD that host uses is unique in that PD0→ PMU0_Buck5, PD1→ PMU0_Buck8, PD2→ PMU0_LDO3 ... etc.
- Per device, the PD→ Buck/LDO mapping is fixed for a particular system and programmed (if needed) by OTP
- Single DVC commands are send from Host using PD & translated to local Buck/LDO on the device. DVC_done irq uses the PD# as the offset that is used for the particular Buck/LDO on device.
- Group DVC that spans multiple devices is not supported. So, if a device gets a group DVC and only one of the PD is mapped on device, it will generate a DVC_error to the host.

Multiple devices, even if not on the same SPMI bus, cannot use the same irq ID for dvc_done messages to the Host.

1.5.5.1 Rules for the DVC Flow Function

1. The slave will not generate the interrupt until the Buck/LDO target has reached the new voltage level. This includes any "Settling time" that is configured for that particular Buck/LDO.
2. Done is defined as a state that the PMU can accept a new command for that Buck/LDO.
 - a. For power up transitions (DVC up), these will mean waiting for the transition to complete
 - b. For power down transitions (DVC down); it can be configured to be after transition is completed or can be configured to be pre-empted "ready for next command". This will be specified in the ERS/configured in OTP for a particular system.
3. The slave can expect that for a particular Buck/LDO, the master will not issue a new DVC request until it sees the "DVC_Done" interrupt for an outstanding DVC request.

Interrupts [223 & 239] is reserved for the slave to indicate to the Host that an error has happened during the DVC transition request. Examples of these errors are

1. Slave received a new DVC request from Host before it indicated that a pending DVC transition has finished.
2. The Buck/LDO does not have DVC enabled.
3. The power state of the PMU disables the Buck/LDO and so DVC is not possible for that Buck.

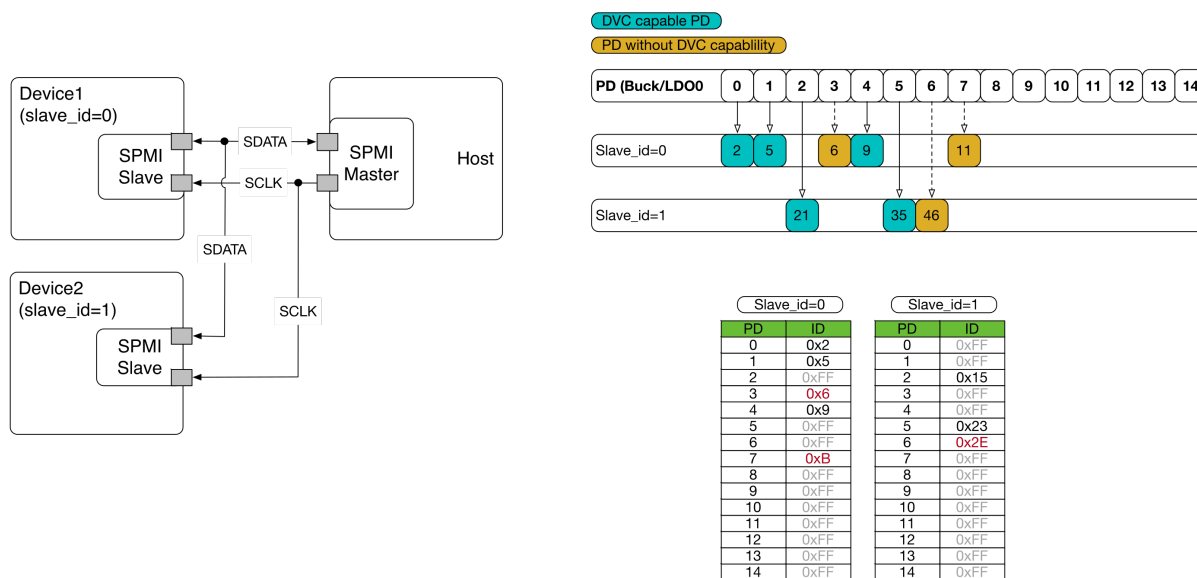
If any of these error conditions occur, the slave is expected to

1. Log the Error in appropriate status registers.
2. Generate a “DVC_Error” write to the master as shown in the earlier tables.

Note: Appendix has details of “Early_Done” DVC transitions.

In a system with multiple DVC capable slaves on the same SPMI bus, each slave will contain a set of registers that lets the Master know how the DVC capable slave PD (Buck/LDO) are mapped to the SPMI PD assignment.

Figure 1.7. Example of multi-slave DVC mapping



In the example above, only 5 DVC capable PD exist in the system, 3 in slave_id=0 (0,1,4) and 2 in slave_id=1 (2,5). All other register configurations have 0xff (invalid PD) or the ERS for the device can specify if PD that are not DVC capable are mapped (shown in red) for those slots.

1.5.6 Interrupt handling

There are a few scenarios that need special mention

1. It is possible for multiple agents to ask the slave to send the same interrupt. There is no H/W based coordination that needs to be done. This will be managed in S/W
2. A single agent can get multiple interrupts simultaneously, there is no expectation of atomicity between interrupts. i.e.; if configured, a pending interrupt should be sent to the Master as a new SPMI message irrespective of Master updates to other register bits.
3. The S/W is responsible for masking and preventing slave from sending additional interrupts before the last one is serviced. Slave H/W should not drop any interrupts i.e.; they should always update the local copy of the interrupt register.

Interrupt sources can be configured as edge sensitive or level sensitive, and typically have three register bits associated with each interrupt:

1. Status (the actual level at the input)
2. Event (set by the device to indicate an interrupt event has been detected), and
3. Mask (if asserted, prevents a MASTER_WRITE sequence from being written to the Master).

1.5.6.1 Edge Sensitive

Upon input edge detection, the event bit is set and if the mask bit=0, the device generates a MASTER_WRITE interrupt sequence. If the sequence completes with an ACK then the interrupt event bit is cleared by the device. The master may set or clear the mask bit at any time. The slave device can be configured to sets its edge sensitivity as rising/falling/either edge, the expected behavior is the same.

1.5.6.2 Level Sensitive

Some interrupt sources (e.g., GPIOs) can be configured as level sensitive. Upon level assertion, the event bit is set and if the mask bit=0, the device generates a MASTER_WRITE interrupt sequence.

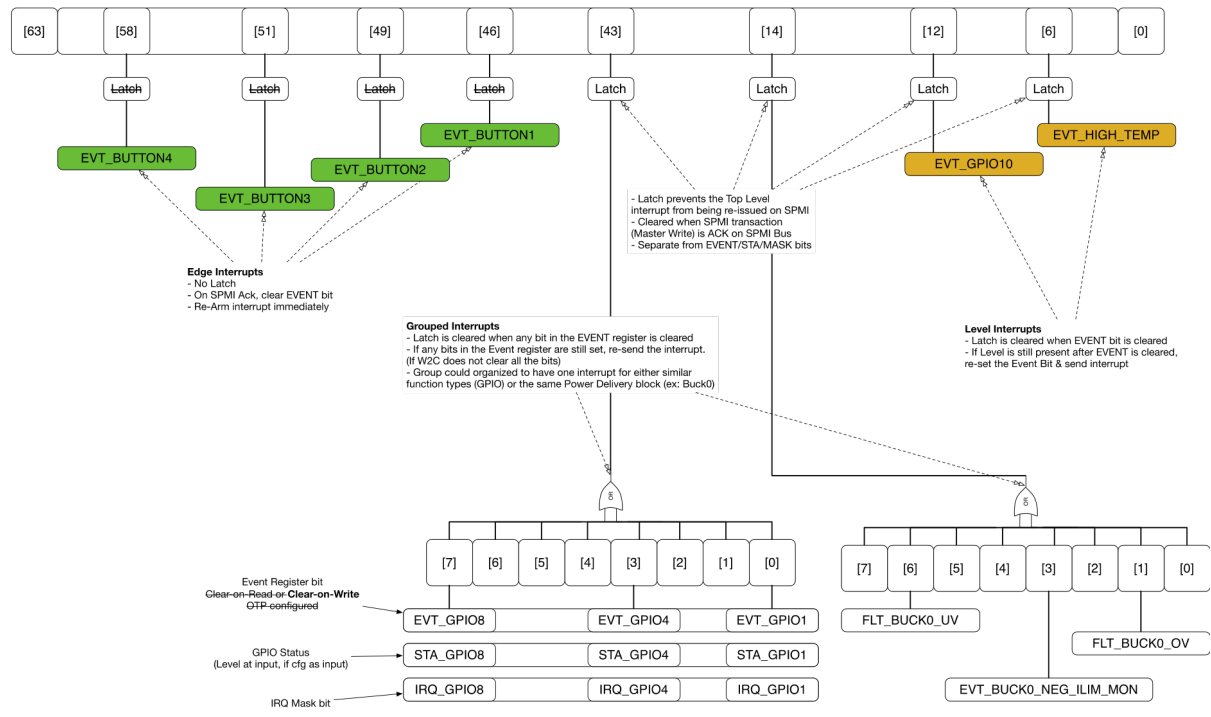
The Latch is cleared when Host SW clears the EVENT bit. If a level interrupt is still present after the EVENT bit is cleared, the device will re-send the interrupt to the Host.

Note that the level at the interrupt input is a sticky bit that is set whenever the status is high. It needs to persist even if the status subsequently goes low. Host software is always responsible to clear the event bit. The Master may set or clear the mask bit at any time.

An example of the interrupt grouping scheme is shown in the next figure.

The details of which interrupts bit will be “Top” level (aka Bit interrupts) vs. those that are in groups (aka Byte interrupts) will be specified in the ERS/configured in OTP for the system.

Figure 1.8. Example of Device Interrupt grouping



Each interrupt will either be a top level interrupt or in one of the groups.

The green interrupts in the figure above are Edge interrupts and the yellow ones are Level.

If multiple bits are set in the same or different registers and are part of the same group, a single group event interrupt is sent to the Host.

The host can clear these bits with either a single burst W1C (write 1 clear) to all the bits that are set or multiple writes to individual registers. The behavior from the slave device should be the same in both the scenarios.

If all bits are cleared, only events that arrive after the W1C should generate a new event.

If any of the bits are not cleared, the slave shall generate another “group” event to the Master.

1.5.6.3 Interrupt Priority

Each device interrupt can be individually configured into one of two priority classes: High or Low.

An interrupt configured as High priority shall be sent with the “A” bit set during the SPMI slave arbitration, and one that is configured as Low priority shall sent with the “SR” bit.

As per SPMI spec, High priority (A-bit set) has priority over Master access as per the SPMI specification.

Each interrupt’s priority level shall be loaded from either SW or OTP memory. The Master can overwrite the priority level with a write to the interrupt’s configuration register.

An event that has been classified as SR priority cannot generate a transaction with A-bit priority set and vice versa.

Priority for each of the interrupts is set to A-bit or SR-bit based on SW/OTP configuration. There is no expectation from the Host on the Queuing or Arbitration in the slave for interrupt ordering. There is an expectation that the SPMI transaction priority will be honored for all transactions. In order of high → low priority, the order as explained in MIPI spec section 8.1 is:

1. (Highest): Slave Priority using Alert (A-bit)
2. Master Priority (A-bit)
3. Slave Secondary (SR-bit)
4. (Lowest) Master Secondary (SR-bit)

1.6 Short Addressing Modes

The SPMI specification supports Short (5b) addressing modes for Register Reads (command: 0x60 – 0x7F) and Register Writes (command: 0x40 – 0x5F). These transactions have a single 8b data payload. To utilize the full extent of this addressing mode, there is additional support added in the target address for these transactions.

Address[4:3] are used as Address pointers to use the 5b short addresses as

'00: Extended Read/Write Address

'01 – '11: used to target 3 different Address pointers (16b Addresses) that can be dedicated to three separate SW agents for handling.

Table 1.13. Short Addressing Mode: Pointers

A[4]	A[3]	A[2]	A[1]	A[0]	Target Address
0	0	0	0	0	Extended Read Address MSB
0	0	0	0	1	Extended Write Address MSB
0	0	0	1	0	AddrPointer1[7:0] & 0xF8
0	0	0	1	1	AddrPointer1[15:8]
0	0	1	0	0	AddrPointer2[7:0] & 0xF8
0	0	1	0	1	AddrPointer2[15:8]
0	0	1	1	0	AddrPointer3[7:0] & 0xF8
0	0	1	1	1	AddrPointer3[15:8]
0	1	0	0	0	AddrPointer1[15:3] Addr[2:0]
0	1				...
0	1	1	1	1	AddrPointer1[15:3] Addr[2:0]
1	0	0	0	0	AddrPointer2[15:3] Addr[2:0]
1	0				...
1	0	1	1	1	AddrPointer2[15:3] Addr[2:0]

A[4]	A[3]	A[2]	A[1]	A[0]	Target Address
1	1	0	0	0	AddrPointer3[15:3] Addr[2:0]
1	1				...
1	1	1	1	1	AddrPointer3[15:3] Addr[2:0]

Note

- Extended Register R/W Long doesn't use this mechanism.
- This short addressing mode is used to target both the lower 8b addresses (using A[4:3] = '00) and target the full 16b register map using the other A[4:3] values. The Address pointers are specified in the SPMI configuration registers.

1.7 Register0 Write

The SPMI specification supports a Register 0 write (command: 0x80 – 0xFF) that uses a 7b payload (Data [6:0]) to signal short commands to the device.

The specific usage for these data bits is specified in the ERS/configured in OTP for the particular device.

Note: Writing to Register Address 0 is not the same as Reg0 command.

1.8 Reset/Sleep/Shutdown/Wakeup

The SPMI specification supports a set of native power state transition commands.

There are two pairs of commands:

Entry and Exit from Low power mode

- Sleep: 0x11. This is a low power mode for the device.
- Wakeup: 0x13. This is the command to exit from a sleep (low power mode) state.

Entry and Exit from Power down

- Reset: 0x10. This command initializes the device.
- Shutdown: 0x12. This is a power down mode for the device and exit is due to a RESET command.

Note: The specific usage for these commands is specified in the ERS for the particular device.

1.9 SGPIO

SGPIO (SPMI-GPIO) is an interface defined between multiple devices in the system. Its goal is two-fold:

- Replace physical wires between devices (virtualize GPIOs)
- Enable multi-device power sequencing without Host intervention.

This new interface enables the System PMU (SGPIO Master) to sequence both Power Delivery IP (bucks, LDOs) and I/O pins in multiple slaves on the SGPIO bus.

This interface is a sub-set of the SPMI specification. When not explicitly stated, the SGPIO spec should refer to the MIPI SPMI spec.

This interface is *in addition* to the SoC (Host) SPMI interface to the PMU.

Unlike the Host_SPMI RCS, the SGPIO SPMI slave is not a RCS (Request Capable Slave). I.e; there is no requirement for the SGPIO slave interface to arbitrate for the bus and to generate interrupts to the SGPIO Master.

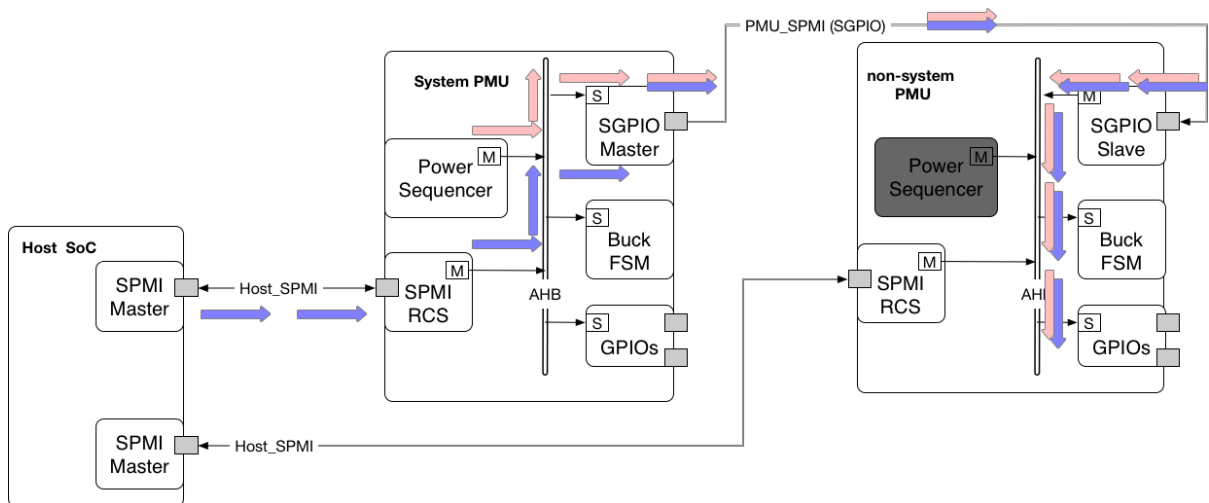
Each slave on the SGPIO interface will have the ability to receive vGPIOs commands from the SGPIO master based ERWL commands from the SGPIO master.

The SGPIO bus has several limitations/restrictions to simplify arbitration between multiple SPMI inputs in the device:

- Only a small set of the SPMI commands are supported (Extended Register Write Long)
- Only specific registers can be accessed using this interface. [0x500 —> 0x51F]
- The data packet consists of the vGPIO (virtual_GPIO) number and a bit to enable or disable it. Each vGPIO can be mapped to one or many of the PDs in the PMU

An example of the system configuration with SGPIO is shown in the figure below.

Figure 1.9. SGPIO interface - system



This example shows the Host with 2 SPMI slaves in the system, one is the system PMU (SGPIO master) and the other is a SGPIO slave. There are 2 paths to access a resource in the non-system PMU (SGPIO slave).

The first path is through the power sequencer in the system PMU (SGPIO master). This sequencer, depending on the power state transition arc and configuration in the system can issue one of more vGPIO “virtual GPIOs” to the slave SGPIO device. It can also issue the command as a broadcast command if it needs to access multiple slave SGPIO devices with the same vGPIO.

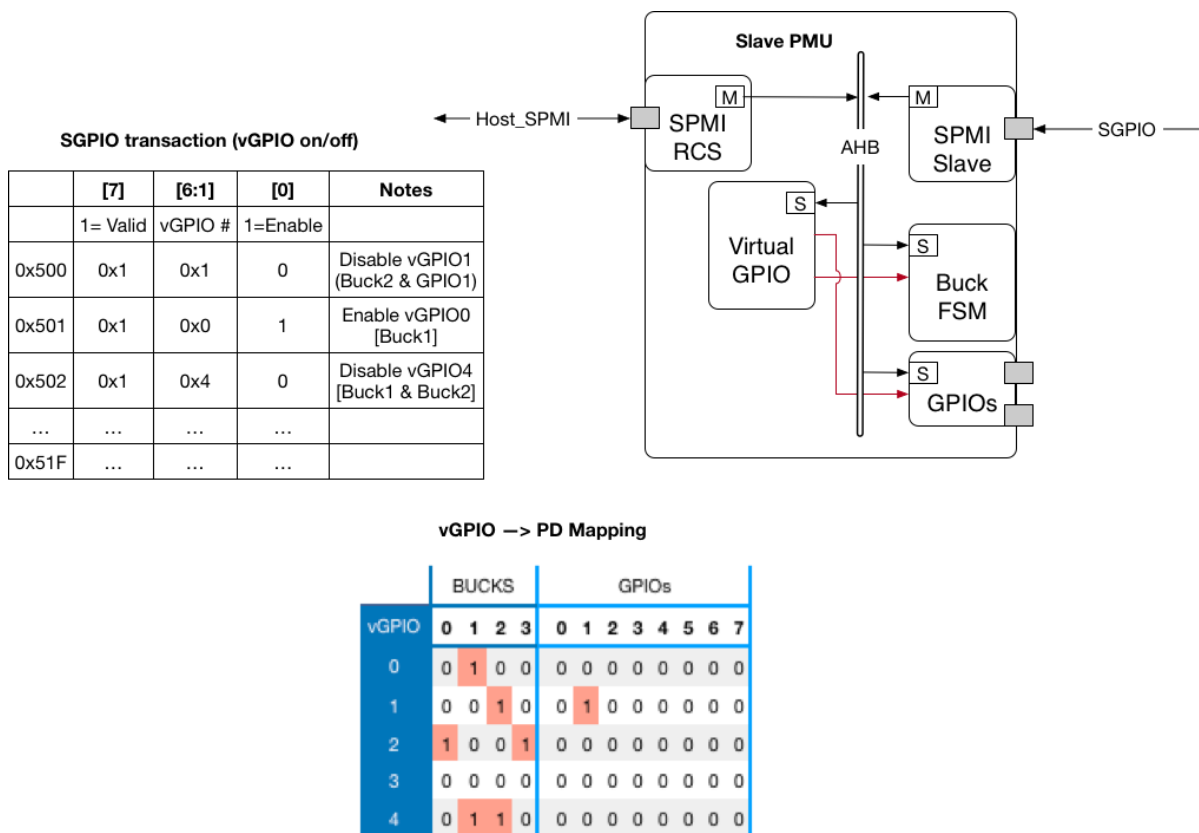
The second path (shown in blue above) is a path where the Host can use the SGPIO_CMD register to access a resource in the SGPIO slave device. This is enabled using the same SGPIO interface from the system (SGPIO master) to the SGPIO slave.

Since there are two paths for the same vGPIO resource in the slave, there is a register SGPIO_CMD_STATUS that allows the Power Sequencer to use this resource un-interrupted by the Host during one of its power state transitions.

The transactions on the SGPIO bus are write (ERWL) transactions that can set/clear any of the vGPIO in the slave device. As configured in the slave device, this vGPIO would map into one or more internal function in the device ie; GPIO configured as GPO, dedicated pin (NRESET, LPM), Buck, LDO, Switch etc.

The SGPIO interface virtualizes a up to 63 GPIOs or supply enables. During power sequencing, the system PMU sends out a SPGPIO enable or disable command at various OTP programmed times (slots) in its sequence. These commands may also be sent as SPMI broadcast commands to all slaves on the SGPIO bus. An example configuration is shown below:

Figure 1.10. SGPIO - Slave Config



The vGPIO → Buck/GPIO mapping is configured in the slave OTP, the actual mapping of vGPIO → PD/GPIO resource is specified in the ERS/configured in OTP for the system. The figures tables below show details of the registers and examples of vGPIO → Buck/GPIO configuration.

The table above shows the SGPIO configuration registers in the SGPIO master (0x500 → 0x51F) that are used to generate transactions on the bus. In the example above,

- vGPIO_1 is disabled @ address 0x500, this translates to Buck2 & GPIO_1
- vGPIO_0 is enabled @ address 0x501, this translates to Buck1
- vGPIO_4 is disabled @ address 0x502, this translates to Buck1 & Buck2

The other addresses have are not sent out on SGPIO (Valid = 0)

An example of the power sequencer & SGPIO usage is shown in the figure below:

Figure 1.11. SGPIO - Power Sequencer

	BUCKS				GPIOs							
vGPIO	0	1	2	3	0	1	2	3	4	5	6	7
0	0	1	1	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	1	0	0
5	1	0	0	1	0	0	0	0	0	0	0	0
6	0	1	0	0	0	0	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	0	0	0	0

		slot											
		U0	U1	U2	U3	U4	U5	U6	U7				
OFF → AWAKE	Buck0	0	0	0	0	0	1	1	1	1	1	1	
	Buck1	0	1	1	1	1	1	1	1	1	1	1	
	Buck2	0	1	1	1	1	1	1	1	1	1	1	
	Buck3	0	0	0	0	0	0	0	1	1	1	1	
	GPIO5	0	0	0	0	0	0	0	0	0	1	1	
vGPIO		0				2		3		4			
SGPIO		1000_0001				1000_0101		1000_0111		1000_1001			
		D7	D6	D5	D4	D3	D2	D1	D0				
AWAKE → OFF	Buck0	1	1	1	1	0	0	0	0	0	0	0	
	Buck1	1	1	0	0	0	0	0	0	0	0	0	
	Buck2	1	1	1	1	1	1	1	0	0	0	0	
	Buck3	1	1	1	1	0	0	0	0	0	0	0	
	GPIO5	1	0	0	0	0	0	0	0	0	0	0	
vGPIO		4				5		7					
SGPIO		1000_1000				1000_1100		1000_1010		1000_1110			

The figure above shows two power sequences with 8 slots each. As an example, the various Bucks/GPIO are enabled/disabled in various slots based on system usage. The SGPIO slave has configuration to map its resources (Buck/GPIO) to one or more vGPIOs as shown in the table on the left.

Based on the power state transition sequence, master determines which resources in the slave need to be toggled and can generate one or more SGPIO transactions per slot

Example above:

- Slot U1 has both Buck1 & Buck2 being enabled, this is mapped to vGPIO_0 in the slave, the SGPIO transaction is "1000_0001" [En,VGPIO=0, Enable]
- Slot D4 has Buck0 & Buck3 being disabled - this is int as "1000_1010" [En, VGPIO=5, Disable]

An example of the configuration table is shown below

In addition to power sequencer usage, the Host can also use this interface to set/clear resources in the slave device by writing to the SGPIO master. To enable this, the following registers are available in the SGPIO master.

Register (RW)	Address	7	6	5	4	3	2	1	0
SGPIO_CMD_ADDR	0x508					Slave_ID_Address			
SGPIO_CMD_DATA	0x509		vGPIO number						1 = Enable, 0 = Disable
SGPIO_CMD_DATA2	0x50A	vGPIO Data							
SGPIO_CMD_EXEC	0x50B								1 = Go
SGPIO_CMD_STATUS	0x50C							1 = ACK, 0 = NACK	1 = Busy, 0 = Idle

Here is a description of the registers:

- SGPIO_CMD_ADDR: This contains the SGPIO slave ID that the vGPIO is targeted for.
- SGPIO_CMD_DATA: This 8b data is the same format as the data sent on SGPIO when used in the power sequencer
- SGPIO_CMD_DATA2: This is optional 1byte data packet that can be set by Master to the slave vGPIO. It could be configuration like PU/PD/OD for a GPIO or Target VSEL for a PD.
- SGPIO_CMD_EXEC: This register has the "GO" bit that is set by Host when it wants the SGPIO master to send the transaction on the SGPIO bus.
- SGPIO_CMD_STATUS: This register has the status of the SGPIO command for information to the Host.

When the power sequencer is using the SGPIO interface, the SGPIO is not available for the Host. This is indicated to the Host by STATUS bit [Busy=1].

The flow control for this usage [Host wants to set vGPIO3=1 on SGPIO slave_ID=5]

1. Host writes to SGPIO_CMD_ADDR=0x5, and SGPIO_CMD_DATA='1000_0111
2. Host polls to check if the bus is idle
 - a. waits until (STATUS[0]=0)
3. Host writes the SGPIO_CMD_EXEC register [0]=1 to enable the transaction over SGPIO
4. Host polls for completion
 - a. wait until (STATUS[1]=1) or IRQ from SGPIO Master on receiving a NACK over SGPIO

The atomicity of the power sequencer using the SGPIO interface is maintained by the SGPIO master. When sending the SGPIO_CMD transaction over the SGPIO bus, the SGPIO Master uses address 0x510 - 0x511. These data packets are available in the 0x510 - 0x511 registers in the slave device (for debug). For example: after the command above has been executed by the SGPIO master, the Host can read from 0x510 in the slave (slave_ID=5) and should get a value of 0x87

The Host SGPIO usage should never interfere with the power sequencer. Once the power sequencer starts, it sets the Busy bit in the STATUS register and will not clear it until the sequence has finished.

Also, if the power sequencer has not started (bus=idle) and Host writes a command, the sequencer will start after the Host command has finished.

The register space for SGPIO is assigned as follows:

0x500 - 0x507 : for the Master Power Sequencer. SGPIO transactions initiated by the Master Power Sequencer will use these addresses on the SGPIO bus.

0x508 - 0x50F : for Host to Master (SGPIO_CMD). These are the registers that the Host SoC will write to the Master to initiate a SGPIO command to the slave.

0x510 - 0x517 : for Host (Master to Slave), SGPIO_CMD transactions from Master to Slave will use these addresses on the SGPIO bus.

0x518 - 0x51F : Reserved

Since multiple PD/GPIOs can be enabled in a particular power sequencer slot, the SGPIO support broadcast using the Group_Slave_ID to set multiple vGPIOs in a single transaction. The ACK/NACK behaviour is different for directed vs. group commands. For directed messages, the target device needs to assert the SDATA pin to NACK a request, for group messages, the slave that wants to NACK has to de-assert the SDATA pin. Since the SGPIO slave is not RCS capable, this ACK/NACK behavior is translated into the CMD_STATUS register by the SGPIO master.

Note: The specific usage for SGPIO configuration and PD/GPIO mappings is specified in the ERS/configured in OTP for the particular device.

1.10 Registers

This section goes over the registers that are required in the Slave for being compliant with the SPMI specification, it is a minimum set of registers that are needed to support the features described in the earlier sections.

Here is the address register map.

Table 1.14. Register Map

Address Range		Description
Start	End	
0x0000	0x000F	Vendor/Application Specific Registers
0x001F	0x001F	SPMI 0-Latency Register Address
0x0020	0x00FF	Vendor/Application Specific Registers
0x0100	0x01FF	SPMI configuration
0x0200	0x02FF	(System PMU specific) Power State Registers
0x0300	0x03FF	(DVC capable PMU specific) DVC Voltage Select Registers
0x0400	0x04FF	(DVC group capable PMU specific) DVC_GROUP Registers
0x0500	0x05FF	(Master/Slave PMU specific) SGPIO Registers
0x0600	0x0FFF	RESERVED
0x1000	0xFFFF	Vendor/Application Specific Registers

Vendor Specific Registers are not covered in this spec, it is shown to let the vendor designs know which address range is available for the device specific registers.

Reserved should not be used, it is left for future use.

The next sections go over each of the register ranges.

1.10.1 SPMI Configuration

There is no enable/disable bit for SPMI, since it is desired that this interface is the only register read/write path between the Master and Slave.

Only registers in the address range (0x100 – 0x10F) are always available to the Master to read within the 2.5 cycles during a register read transaction (aka 0-Latency Read Registers)

Table 1.15. SPMI Configuration

Register	Type	Address	Default	7	6	5	4	3	2	1	0
SPMI_0LAT_REG_ADDR	RO ¹	0x01F	0 = 0x100	ADDRESS_OF_0_LATENCY_REGISTERS = [0LAT_ADDR]							
SPMI_CTRL	RW ¹	[0LAT_ADDR]	0x1F				RCS_EN	SLAVE_ID			
SPMI_REVISION	RO ¹	[0LAT_ADDR]+1	0x03							SPMI_REVISION	
MASTER_HOST_ID	RW ¹	[0LAT_ADDR]+2	0x00							HOST_MASTER_ID	
READ_LATENCY	RO ¹	[0LAT_ADDR]+3	0x18	READ_LATENCY							

INTERRUPT_ADDR	RW ¹	[0LAT_ADDR]+4	0x00	MASTER_WRITE_ADDRESS							
INTERRUPT_OFFSET	RW ¹	[0LAT_ADDR]+5	0x00	OFFSET_VALUE							
DEVICE_CAPABILITY_TYPE	RO ¹	[0LAT_ADDR]+6	0x00				LDO_DVC	Slave-to-slave	DVC group	Buck_DVC	System PMU
DEVICE_CAPABILITY_CMD	RO ¹	[0LAT_ADDR]+7	0x00					Native Power	Reg0	Backlight	SAM
INTERRUPT_VW_ADDR	RW ¹	[0LAT_ADDR]+8	0x01	MASTER_WRITE_ADDRESS_FOR_VIRTUAL_WIRES							
MASTER_PMU_ID	RW ¹	[0LAT_ADDR]+9	0x01							PMU_MASTER_ID	
SGPIO_SLAVE_ID	RW ¹	[0LAT_ADDR]+10	0x0					SGPIO_SLAVE_ID			
GROUP_SLAVE_ID	RW ¹	[0LAT_ADDR]+11	0x1F				GSID_EN	GROUP_SLAVE_ID			
		0x112 – 0x11F		RESERVED							
EXT_READ_ADDRESS	RW	120	0x00	8b Address for Short Addressing Mode support Extended Read Address							
EXT_WRITE_ADDRESS	RW	121	0x00	8b Address for Short Addressing Mode support Extended Write Address							
ADDRPOINTER1_LOW	RW	122	0x00	Lower 8b of Address <i>Pointer1</i> Base for Short Addressing Mode support							
ADDRPOINTER1_HIGH	RW	123	0x00	Upper 8b of Address <i>Pointer1</i> Base for Short Addressing Mode support							
ADDRPOINTER2_LOW	RW	124	0x00	Lower 8b of Address <i>Pointer2</i> Base for Short Addressing Mode support							
ADDRPOINTER2_HIGH	RW	125	0x00	Upper 8b of Address <i>Pointer2</i> Base for Short Addressing Mode support							
ADDRPOINTER3_LOW	RW	126	0x00	Lower 8b of Address <i>Pointer3</i> Base for Short Addressing Mode support							
ADDRPOINTER3_HIGH	RW	127	0x00	Upper 8b of Address <i>Pointer3</i> Base for Short Addressing Mode support							
		0x128 – 0x13F		RESERVED							
ARB_MON_TIMEOUT	RW	140	0x1F				ARB_TO_EN	ARB_MON_TIMEOUT			
CMD_SEQ_TIMEOUT	RW	141	0x1E				CMD_TO_EN	CMD_SEQ_TIMEOUT			
		0x142 - 0x15F		RESERVED							
ARB_REQ_HOLD_CTRL	RW	160	0x00							A_HOLD_EN	SR_HOLD_EN
ERROR_CTRL	RW	161	0xC					NACK_RETRY_CNT		NACK_CRASH_RESET_EN	TO_CRASH_EN
ERROR_EVENT	RW	162	0x00	INTERNAL_ERR	INT_BUS_ERR	NACK_ON_MWRITE	UNKNOWN_CMD	CMD_PARITY	ADDR_PARITY	DATA_PARITY	SCLK_TIMEOUT
ERROR_MASK	RW	163	0x00	INTERNAL_ERR	INT_BUS_ERR	NACK_ON_MWRITE	UNKNOWN_CMD	CMD_PARITY	ADDR_PARITY	DATA_PARITY	SCLK_TIMEOUT
		0x164 - 0x16E		RESERVED							
IRQ_ENABLE	RW	16F	0x00								IRQ_ENABLE
EVT_Prio_REG_EVT7_EVT0	RW	170	0x00	EVT7	EVT6	EVT5	EVT4	EVT3	EVT2	EVT1	EVT0
EVT_Prio_REG_EVT15_EVT8	RW	171	0x00	EVT15	EVT14	EVT13	EVT12	EVT11	EVT10	EVT9	EVT8
EVT_Prio_REG_EVT23_EVT16	RW	172	0x00	EVT23	EVT22	EVT21	EVT20	EVT19	EVT18	EVT17	EVT16
EVT_Prio_REG_EVT31_EVT24	RW	173	0x00	EVT31	EVT30	EVT29	EVT28	EVT27	EVT26	EVT25	EVT24
EVT_Prio_REG_EVT39_EVT32	RW	174	0x00	EVT39	EVT38	EVT37	EVT36	EVT35	EVT34	EVT33	EVT32
EVT_Prio_REG_EVT47_EVT40	RW	175	0x00	EVT47	EVT46	EVT45	EVT44	EVT43	EVT42	EVT41	EVT40
EVT_Prio_REG_EVT55_EVT48	RW	176	0x00	EVT55	EVT54	EVT53	EVT52	EVT51	EVT50	EVT49	EVT48
EVT_Prio_REG_EVT63_EVT56	RW	177	0x00	EVT63	EVT62	EVT61	EVT60	EVT59	EVT58	EVT57	EVT56
EVT_Prio_REG_EVT71_EVT64	RW	178	0x00	EVT71	EVT70	EVT69	EVT68	EVT67	EVT66	EVT65	EVT64

EVT_PRIO_REG_ EVT79_ EVT72	RW	179	0x00	EVT79	EVT78	EVT77	EVT76	EVT75	EVT74	EVT73	EVT72
EVT_PRIO_REG_ EVT87_ EVT80	RW	17A	0x00	EVT87	EVT86	EVT85	EVT84	EVT83	EVT82	EVT81	EVT80
EVT_PRIO_REG_ EVT95_ EVT88	RW	17B	0x00	EVT95	EVT94	EVT93	EVT92	EVT91	EVT90	EVT89	EVT88
EVT_PRIO_REG_ EVT103_ EVT96	RW	17C	0x00	EVT103	EVT102	EVT101	EVT100	EVT99	EVT98	EVT97	EVT96
EVT_PRIO_REG_ EVT111_ EVT104	RW	17D	0x00	EVT111	EVT110	EVT109	EVT108	EVT107	EVT106	EVT105	EVT104
EVT_PRIO_REG_ EVT119_ EVT112	RW	17E	0x00	EVT119	EVT118	EVT117	EVT116	EVT115	EVT114	EVT113	EVT112
EVT_PRIO_REG_ EVT127_ EVT120	RW	17F	0x00	EVT127	EVT126	EVT125	EVT124	EVT123	EVT122	EVT121	EVT120
EVT_PRIO_REG_ EVT135_ EVT128	RW	180	0x00	EVT135	EVT134	EVT133	EVT132	EVT131	EVT130	EVT129	EVT128
EVT_PRIO_REG_ EVT143_ EVT136	RW	181	0x00	EVT143	EVT142	EVT141	EVT140	EVT139	EVT138	EVT137	EVT136
EVT_PRIO_REG_ EVT151_ EVT144	RW	182	0x00	EVT151	EVT150	EVT149	EVT148	EVT147	EVT146	EVT145	EVT144
EVT_PRIO_REG_ EVT159_ EVT152	RW	183	0x00	EVT159	EVT158	EVT157	EVT156	EVT155	EVT154	EVT153	EVT152
EVT_PRIO_REG_ EVT167_ EVT160	RW	184	0x00	EVT167	EVT166	EVT165	EVT164	EVT163	EVT162	EVT161	EVT160
EVT_PRIO_REG_ EVT175_ EVT168	RW	185	0x00	EVT175	EVT174	EVT173	EVT172	EVT171	EVT170	EVT169	EVT168
EVT_PRIO_REG_ EVT183_ EVT176	RW	186	0x00	EVT183	EVT182	EVT181	EVT180	EVT179	EVT178	EVT177	EVT176
EVT_PRIO_REG_ EVT191_ EVT184	RW	187	0x00	EVT191	EVT190	EVT189	EVT188	EVT187	EVT186	EVT185	EVT184
EVT_PRIO_REG_ EVT199_ EVT192	RW	188	0x00	EVT199	EVT198	EVT197	EVT196	EVT195	EVT194	EVT193	EVT192
EVT_PRIO_REG_ EVT207_ EVT200	RW	189	0x00	EVT207	EVT206	EVT205	EVT204	EVT203	EVT202	EVT201	EVT200
EVT_PRIO_REG_ EVT215_ EVT208	RW	18A	0x00	EVT215	EVT214	EVT213	EVT212	EVT211	EVT210	EVT209	EVT208
EVT_PRIO_REG_ EVT223_ EVT216	RW	18B	0x00	EVT223	EVT222	EVT221	EVT220	EVT219	EVT218	EVT217	EVT216
EVT_PRIO_REG_ EVT231_ EVT224	RW	18C	0x00	EVT231	EVT230	EVT229	EVT228	EVT227	EVT226	EVT225	EVT224
EVT_PRIO_REG_ EVT239_ EVT232	RW	18D	0x00	EVT239	EVT238	EVT237	EVT236	EVT235	EVT234	EVT233	EVT232
EVT_PRIO_REG_ EVT247_ EVT240	RW	18E	0x00	EVT247	EVT246	EVT245	EVT244	EVT243	EVT242	EVT241	EVT240
EVT_PRIO_REG_ EVT255_ EVT248	RW	18F	0x00	EVT255	EVT254	EVT253	EVT252	EVT251	EVT250	EVT249	EVT248
		0x190 - 0x1FF		RESERVED							

¹0latency

1.10.1.1 SPMI_0LAT_REG_ADDR

- This register contains the address of the 0-latency registers in the slave IP.
- Default = [value = 0x0; address = 0x100]
- For backwards compatibility, a value of 0x0 in this reg will be interpreted by Host SW as (0x100)
- For 16b address capable devices, the value will be '0 (i.e.; value 0x0 = address 0x100)

- For 8b address capable devices, the value in this register is the address (i.e.; value 0xf = address 0xf)
- This is a read-only register.

1.10.1.2 SPMI_CTRL

- The Slave_ID is the ID for the Device that is either pin-strapped or OTP programmed.
- Host S/W can override this value if needed. This is the value that the slave SPMI controller should use to match transactions and generate ID when using the SPMI bus.
- RCS_EN (Request Capable Slave Enable) is '1' for slave devices that will use the "in-band" interrupt message capabilities shown in earlier sections.
- Note: This is a "capability" register, not the register used to enable interrupts. The IRQ_ENBLE bit is at register @ address 0x170
- This is a read-write register.

1.10.1.3 SPMI_REVISION

- This is the version of the SPMI spec that is supported in the slave. It will be used to distinguish between future revisions of the SPMI spec.
- The value for devices supporting this version of the spec is '03
- This is a read-only register.

1.10.1.4 MASTER_ID

- This is the value that the slave should use when sending "Master Write" interrupts on the SPMI Bus.
- Default value = 0x0.
- This is a read-write register.

1.10.1.5 READ_LATENCY

- SPMI spec requires a slave device to respond to a register read transaction from the Master in 2.5 SCLK cycles. This is cumbersome for devices that don't use 24MHz for internal register accesses paths.
- To facilitate this, the master will add a programmed delay in the read transaction (during the Bus idle cycle) to the slave. The master will do this by stopping SCLK for a programmed amount of time (clocks).
- Each slave device needs to let the Master know of the "additional" time that the slave device needs to get the data ready for the master during a register read. This is the 8b value that is programmed in the slave device. Each bit corresponds to a 24 MHz time period; i.e. 41.7 ns.
- The Host S/W will read this value and program the slave register reads with a value that has at least this value (or greater) during the Bus_Park cycle.

- Recommended value in this register = 0x18 (= 24 = 1us)
- This is a read-only register.

1.10.1.6 INTERRUPT_ADDR

- This is the 8b value that is programmed by Host S/W to let the slave know which address to use for interrupt generation. Default = 0x0
- This is a read-write register.

1.10.1.7 INTERRUPT_OFFSET

- This is the 8b value that is programmed by Host S/W to let the slave know how much offset to add for the device specific interrupt generation.
- An offset implies that the interrupt ID sent on Master Write transactions by the device will adjusted by this value.
 - For ex: if the offset is 0x8, an interrupt value of 0x1 will be sent on the SPMI bus as a value of 0x9
- Default = 0x0
- This is a read-write register.

1.10.1.8 DEVICE_CAPABILITY_TYPE

- This is a READ-ONLY register that is set by the device to let the Master/Host know about the type of capabilities supported in the device.
- This is a read-only register.

Table 1.16. Device capability_type bits

Bit value	Device Capability Type	Notes
[0] = 1	System PMU	Supports System PMU capabilities, including Power State transition done/error messages to Host.
[1] = 1	Buck DVC	Supports Bucks that have DVC capability using the SPMI engine w/ DVC done/error messages to Host.
[2] = 1	DVC group	Supports PDs that have DVC group capability using the SPMI engine w/ DVC group done/error messages to Host.
[3] = 1	Slave to Slave	Supports slave to slave transactions in the device.
[4] = 1	LDO DVC	Supports LDOs that have DVC capability using the SPMI engine w/ DVC done/error messages to Host.
[5] = 1	SGPIO	In addition to the Host interface, this device supports another SPMI interface (SGPIO)
[6] = 0	Reserved	
[7] = 0	Reserved	

A device can have multiple bits set. For example, a system PMU with Buck DVC capability will have bits [0] and [1] set.

A value of all 0s defines the following “Base capabilities” in the IP

- IP is compliant to Rev 0.87 of Apple SPMI spec
- Tracking the SPMI bus and following the command sequence
- Ack/Nack for commands that match the IPs slave_ID
- If RCS=1;
 - Ability to generate request for the Bus using SDATA=1
 - Ability to generate Master_Write transactions using the “Interrupt_Addr” and “Interrupt Offset” fields
- Support read by Host of any registers in the 0-latency register read address space
- For each of the "type" bits that are not set, the device needs to have a configuration to either ignore the command/transaction or generate an error.
- For example: sending a SPS (target_state) register write to a device that has system_PMU=0 will ignore the command (& if configured) generate an Error to the Master.
- If Slave_to_slave=0, the device is not capable to both generate and accept a slave_to_slave transaction on the bus.
- If System_PMU=0, the device should not be capable of any system power state transitions (i.e.; Error response to Target_State register writes)

1.10.1.9 DEVICE_CAPABILITY_CMD

- This is a READ-ONLY register that is set by the device to let the Master/Host know about the SPMI commands that are supported in the device.
- This is a read-only register.

Table 1.17. Device capability_cmd bits

Bit value	Device Capability Command	Notes
[0] = 1	SAM	Supports SAM (Short Addressing Mode) commands.
[1] = 1	Backlight	Supports Backlight (WLED & BLC) commands.
[2] = 1	Reg0	Supports Register0 write commands
[3] = 1	Native power	Supports native (Reset, Sleep, Shutdown, Wake) commands on SPMI interface.
[4] = 1	Display	Supports Display commands
[5] = 1	Virtual wires	Supports Virtual wires interrupt generation (using VW_ADDR)

Bit value	Device Capability Command	Notes
[6] = 0	Reserved	
[7] = 0	Reserved	

A device can have multiple bits set. For example, a device that supports both SAM and backlight commands will have bits [0] and [1] set.

1.10.1.10 INTERRUPT_VW_ADDR

- This is the 8b value that is programmed by Host S/W to let the slave know which address to use for "Virtual Wire" interrupt generation.
- Default = 0x1
- This is a read-write register.

1.10.1.11 MASTER_PMU_ID

- If the device supports a Master SPMI interface for communicating with other slave devices, this is the 2b value that is programmed by Host S/W to let the device know which address to use for its SPMI transactions.
- Default = 0x1
- This is a read-write register.

1.10.1.12 SGPIO_SLAVE_ID

- If the device supports the system PMU Master/Slave SPMI interface in addition to the Host SPMI interface, this 8b value is programmed by Host S/W to let the slave know which address to use for the slave on SGPIO.
- Default = 0x0
- This is a read-write register.

1.10.1.13 GROUP_SLAVE_ID

- This register is used to configure "group" commands from Master to Slave devices in the system.
- The Host SW will program this register to send group commands to all devices that have GSID enabled and those that match the Group_Slave_ID
- Default = 0x1F
- This is a read-write register.

1.10.1.14 EXT_{READ,WRITE}_ADDRESS

- These are the addresses used by the device SAM (Short Addressing Mode) support.

- These registers are the values that are used by the Master to use the 8b target address for the Short Addressing Mode support.
- Details of usage are described in earlier sections.
- Default = 0x0
- This is a read-write register.

1.10.1.15 ADDRPOINTER{1,2,3}_{LOW,HIGH}

- These are the address pointers used by the device SAM (Short Addressing Mode) support.
- This is the value that is used by the Master to use a different 16b target address for the Short Addressing Mode support.
- Details of usage are described in earlier sections.
- Default = 0x0
- This is a read-write register.

1.10.1.16 ARB_MON_TIMEOUT

- ARB_MON_TIMEOUT is the timer value that starts when SCLK is inactive. It is used to determine if the SCLK input line has no activity for a programmable period of time.
- If it expires without SCLK activity, the slave can start a new arbitration request by setting SDATA high. It restarts when SCLK activity is detected.
- Default = 0xF (96 us)
- This is a read-write register.

1.10.1.17 CMD_SEQ_MON_TIMEOUT

- CMD_SEQ_TIMEOUT is the timer value that starts when SCLK is active. It is used to determine if the SCLK input line has no activity for a programmable period of time.
- If there is no activity in the SPMI clock and the SPMI slave FSM is not in the IDLE state, it will time-out and go to IDLE state after this timer expires and reset the slave interface.
- Default = 0xE (80us)
- This is a read-write register.

1.10.1.18 ARB_REQ_HOLD_CTRL

- ARB_REQ_HOLD_CTRL is a configuration register that enables slave hold request functionality.
- There are separate bits for A-type and SR-type transactions.
- Default = 0
- This is a read-write register.

1.10.1.19 ERROR_CTRL

- ERROR_CTRL is a configuration register that contains error handling capability.
- TO_CRASH_EN: This bit allows the device to signal CRASH to system PMU on timeout.
- NACK_CRASH_EN: This bit allows the device to signal CRASH on getting a NACK on the SPMI bus.
- NACK_ENTRY_CNT: This is the number of retries before the device registers a 'NACK Error'. Default = 3
- Default = 0xC
- This is a read-write register.

1.10.1.20 IRQ_ENABLE

- IRQ_ENABLE is a bit in the register that enables the RCS capability in the device.
- Default = 0x0
- This is a read-write register.

1.10.1.21 EVT_PRIO_REG_EVT{0 – 255}

- These registers are used to set the priority of the events/interrupts that are sent from the device to the Host. Each bit is set for "A/SR" priority for each of the events.
- For ex: If Reg_evt7_evt0[7:0] = 0x00001011, that means that events (0,1,3) are going to be sent with A-bit and events (7,6,5,4,2) will be sent with SR bit.
- The Host SW will program these register based on system requirements.
- Default = 0x0
- This is a read-write register.

1.10.2 Power State and DVC Registers⁴

The Power sequencing FSM in the Slave/PMIC has a couple of registers that it uses for the power state sequencing.

The Host SW is supposed to check the capabilities register before accessing the Power State Registers in a slave device. If the Host SW generates a write access to the virtual address space for Power_State Registers (0x201 – 0x202) for a slave that is not a "System PMU", the device will generate an "Invalid State Transition" Error back to the Host.

It is okay for the slaves to drop write accesses to UN-implemented registers and return reads as 0s. 0x200 is a read-only register, so writes are dropped and reads return 0 for non-system PMUs.

⁴System PMU Specific

Table 1.18. Power State

Register	Type	Address	Default	7	6	5	4	3	2	1	0
POWER_STATE_STATUS	RO	200	0x00	IN_TRANSITION		PREVIOUS_STATE			CURRENT_STATE		
POWER_STATE_TARGET	RW	201	0x00						TARGET_STATE		
POWER_STATE_FORCE	RW	202	0x00	FORCE_VALID					FORCE_STATE		
		0x203 - 0x2FF		RESERVED							
PD0_VSEL_TARGET	RW	300	0x00	PD0 Target Voltage							
PD1_VSEL_TARGET	RW	301	0x00	PD1 Target Voltage							
PD2_VSEL_TARGET	RW	302	0x00	PD2 Target Voltage							
PD3_VSEL_TARGET	RW	303	0x00	PD3 Target Voltage							
PD4_VSEL_TARGET	RW	304	0x00	PD4 Target Voltage							
PD5_VSEL_TARGET	RW	305	0x00	PD5 Target Voltage							
PD6_VSEL_TARGET	RW	306	0x00	PD6 Target Voltage							
PD7_VSEL_TARGET	RW	307	0x00	PD7 Target Voltage							
PD8_VSEL_TARGET	RW	308	0x00	PD8 Target Voltage							
PD9_VSEL_TARGET	RW	309	0x00	PD9 Target Voltage							
PD10_VSEL_TARGET	RW	30A	0x00	PD10 Target Voltage							
PD11_VSEL_TARGET	RW	30B	0x00	PD11 Target Voltage							
PD12_VSEL_TARGET	RW	30C	0x00	PD12 Target Voltage							
PD13_VSEL_TARGET	RW	30D	0x00	PD13 Target Voltage							
PD14_VSEL_TARGET	RW	30E	0x00	PD14 Target Voltage							
PD15_VSEL_TARGET	RW	30F	0x00	PD15 Target Voltage							
PD16_VSEL_TARGET	RW	310	0x00	PD16 Target Voltage							
PD17_VSEL_TARGET	RW	311	0x00	PD17 Target Voltage							
PD18_VSEL_TARGET	RW	312	0x00	PD18 Target Voltage							
PD19_VSEL_TARGET	RW	313	0x00	PD19 Target Voltage							
PD20_VSEL_TARGET	RW	314	0x00	PD20 Target Voltage							
PD21_VSEL_TARGET	RW	315	0x00	PD21 Target Voltage							
PD22_VSEL_TARGET	RW	316	0x00	PD22 Target Voltage							
PD23_VSEL_TARGET	RW	317	0x00	PD23 Target Voltage							
PD24_VSEL_TARGET	RW	318	0x00	PD24 Target Voltage							
PD25_VSEL_TARGET	RW	319	0x00	PD25 Target Voltage							
PD26_VSEL_TARGET	RW	31A	0x00	PD26 Target Voltage							
PD27_VSEL_TARGET	RW	31B	0x00	PD27 Target Voltage							
PD28_VSEL_TARGET	RW	31C	0x00	PD28 Target Voltage							
PD29_VSEL_TARGET	RW	31D	0x00	PD29 Target Voltage							
PD30_VSEL_TARGET	RW	31E	0x00	PD30 Target Voltage							
		0x31F - 0x3FF		RESERVED							
DVC_GROUP_CFG0_0	RW	400	0x00	VALID			PD_A				
DVC_GROUP_CFG0_1	RW	401	0x00	VALID			PD_B				
DVC_GROUP_CFG0_2	RW	402	0x00	VALID			PD_C				
DVC_GROUP_CFG0_3	RW	403	0x00	VALID			PD_D				
DVC_GROUP_CFG1_0	RW	404	0x00	VALID			PD_A				

DVC_GROUP_CFG1_1	RW	405	0x00	VALID			PD_B
DVC_GROUP_CFG1_2	RW	406	0x00	VALID			PD_C
DVC_GROUP_CFG1_3	RW	407	0x00	VALID			PD_D
DVC_GROUP_CFG2_0	RW	408	0x00	VALID			PD_A
DVC_GROUP_CFG2_1	RW	409	0x00	VALID			PD_B
DVC_GROUP_CFG2_2	RW	40A	0x00	VALID			PD_C
DVC_GROUP_CFG2_3	RW	40B	0x00	VALID			PD_D
DVC_GROUP_CFG3_0	RW	40C	0x00	VALID			PD_A
DVC_GROUP_CFG3_1	RW	40D	0x00	VALID			PD_B
DVC_GROUP_CFG3_2	RW	40E	0x00	VALID			PD_C
DVC_GROUP_CFG3_3	RW	40F	0x00	VALID			PD_D
DVC_GROUP_CFG4_0	RW	410	0x00	VALID			PD_A
DVC_GROUP_CFG4_1	RW	411	0x00	VALID			PD_B
DVC_GROUP_CFG4_2	RW	412	0x00	VALID			PD_C
DVC_GROUP_CFG4_3	RW	413	0x00	VALID			PD_D
DVC_GROUP_CFG5_0	RW	414	0x00	VALID			PD_A
DVC_GROUP_CFG5_1	RW	415	0x00	VALID			PD_B
DVC_GROUP_CFG5_2	RW	416	0x00	VALID			PD_C
DVC_GROUP_CFG5_3	RW	417	0x00	VALID			PD_D
DVC_GROUP_CFG6_0	RW	418	0x00	VALID			PD_A
DVC_GROUP_CFG6_1	RW	419	0x00	VALID			PD_B
DVC_GROUP_CFG6_2	RW	41A	0x00	VALID			PD_C
DVC_GROUP_CFG6_3	RW	41B	0x00	VALID			PD_D
		0x41C - 0x43F		RESERVED			
DVC_GROUP_0_VSEL_TARGETA	RW	440	0x00	Target Voltage for PD_A in DVC Group			
DVC_GROUP_0_VSEL_TARGETB	RW	441	0x00	Target Voltage for PD_B in DVC Group			
DVC_GROUP_0_VSEL_TARGETC	RW	442	0x00	Target Voltage for PD_C in DVC Group			
DVC_GROUP_0_VSEL_TARGETD	RW	443	0x00	Target Voltage for PD_D in DVC Group			
DVC_GROUP_1_VSEL_TARGETA	RW	444	0x00	Target Voltage for PD_A in DVC Group			
DVC_GROUP_1_VSEL_TARGETB	RW	445	0x00	Target Voltage for PD_B in DVC Group			
DVC_GROUP_1_VSEL_TARGETC	RW	446	0x00	Target Voltage for PD_C in DVC Group			
DVC_GROUP_1_VSEL_TARGETD	RW	447	0x00	Target Voltage for PD_D in DVC Group			
DVC_GROUP_2_VSEL_TARGETA	RW	448	0x00	Target Voltage for PD_A in DVC Group			
DVC_GROUP_2_VSEL_TARGETB	RW	449	0x00	Target Voltage for PD_B in DVC Group			
DVC_GROUP_2_VSEL_TARGETC	RW	44A	0x00	Target Voltage for PD_C in DVC Group			
DVC_GROUP_2_VSEL_TARGETD	RW	44B	0x00	Target Voltage for PD_D in DVC Group			
DVC_GROUP_3_VSEL_TARGETA	RW	44C	0x00	Target Voltage for PD_A in DVC Group			
DVC_GROUP_3_VSEL_TARGETB	RW	44D	0x00	Target Voltage for PD_B in DVC Group			
DVC_GROUP_3_VSEL_TARGETC	RW	44E	0x00	Target Voltage for PD_C in DVC Group			
DVC_GROUP_3_VSEL_TARGETD	RW	44F	0x00	Target Voltage for PD_D in DVC Group			
DVC_GROUP_4_VSEL_TARGETA	RW	450	0x00	Target Voltage for PD_A in DVC Group			
DVC_GROUP_4_VSEL_TARGETB	RW	451	0x00	Target Voltage for PD_B in DVC Group			
DVC_GROUP_4_VSEL_TARGETC	RW	452	0x00	Target Voltage for PD_C in DVC Group			
DVC_GROUP_4_VSEL_TARGETD	RW	453	0x00	Target Voltage for PD_D in DVC Group			

DVC_GROUP_5_VSEL_TARGETA	RW	454	0x00	Target Voltage for PD_A in DVC Group
DVC_GROUP_5_VSEL_TARGETB	RW	455	0x00	Target Voltage for PD_B in DVC Group
DVC_GROUP_5_VSEL_TARGETC	RW	456	0x00	Target Voltage for PD_C in DVC Group
DVC_GROUP_5_VSEL_TARGETD	RW	457	0x00	Target Voltage for PD_D in DVC Group
DVC_GROUP_6_VSEL_TARGETA	RW	458	0x00	Target Voltage for PD_A in DVC Group
DVC_GROUP_6_VSEL_TARGETB	RW	459	0x00	Target Voltage for PD_B in DVC Group
DVC_GROUP_6_VSEL_TARGETC	RW	45A	0x00	Target Voltage for PD_C in DVC Group
DVC_GROUP_6_VSEL_TARGETD	RW	45B	0x00	Target Voltage for PD_D in DVC Group
		0x45C - 0x4FF		RESERVED

Table 1.19. Power State Registers

Register	POWER_STATE_STATUS	
This register is used by the PMU to inform the Host of the power state that the PMU is in. Steady state expectation is that Current_State = Target_State.		
On transition start from Current_State, the [In_Transition] bit is set and on completion of the transition, the values in [Previous_State] and [Current_State] fields are updated.		
Address	0x200	
Type	READ_ONLY	
Bit	Field	Description
[7]	IN_TRANSITION	1 - State transition has started '0 - Default (No on-going Power State transition)
[6]	RESERVED	
[5:3]	PREVIOUS_STATE	3b encoding of the previous power state of the PMU '000 = Power_State0 ... '111 - Power_State7
[2:0]	CURRENT_STATE	3b encoding of the current power state of the PMU '000 = Power_State0 ... '111 - Power_State7
Register	POWER_STATE_TARGET	
This register is used by the Master to transition the PMU into a particular power state.		
Address	0x201	
Type	READ_WRITE	
Bit	Field	Description
[7:3]	RESERVED	
[2:0]	TARGET_STATE	3b encoding of the target power state for the PMU '000 = Power_State0 ... '111 - Power_State7
Register	POWER_STATE_FORCE	
This register is used to force a transition (and prevent transitions) from/to a particular power_state. If the valid bit is not set, there is no impact of the values in this register.		

Register	POWER_STATE_STATUS	
If the valid bit is set and the [Current_State] of PMU doesn't match [Force_State]; the FSM will transition to [Force_State] and stay there.		
Address	0x202	
Type	READ_WRITE	
Bit	Field	Description
[7]	FORCE_VALID	1 - Enables the "Force_Power_State" field '0 - Default (No force of Power State)
[6:3]	RESERVED	
[2:0]	FORCE_STATE	3b encoding to force the target power state of the PMU '000 = Power_State0 ... '111 = Power_State7

The POWER_STATE_FORCE.FORCE_STATE field is an override of the POWER_STATE_TARGET.POWER_TARGET field.

The value of the [Current_State] register will only get updated to the new state on completion of the transition, so during a power state transition, if the Host does a read of the [Current_State] register, it should get the old value.

1.10.3 DVC⁵

The Host SW is supposed to check the capabilities register before accessing the DVC registers in a slave device. If the Host SW generates a write access to the virtual address space for a PD DVC registers (0x300 – 0x31F) for a slave PD that does not exist in the “PD mapping table” device, the device will generate a “DVC Error” back to the Host. Reads to PD registers, which are not in the “PD mapping table”, generate 0s.

Reads to unimplemented registers (0x320 – 0x3FF) will generate 0s, writes will be dropped, no error is generated.

There are two types of DVC that is supported in this specification.

1. Single rail DVC: Accomplished by a 1 byte (data = 8b, VSEL_TARGET) write from master to slave
2. Multi rail (aka Group) DVC: Used for ramping 2 or more rails together using a single SPMI command.

For the single rail DVC, the host S/W will program the appropriate Buck/LDO in the design with the PD number as used in the SPMI spec.

The actual registers for the Buck/LDO block are unchanged, the PDx_VSEL_TARGET register is used by the host to set a new DVC target for that PD (Buck/LDO).

Table 1.20. DVC VSEL Registers

Register	Address	Data [7:0]
PD0_VSEL_TARGET	0x0300	Buck/LDO0 Target Voltage

⁵DVC capable PMU Specific

Register	Address	Data [7:0]
...		
PD14_VSEL_TARGET	0x030E	Buck/LDO14 Target Voltage
PD15_VSEL_TARGET	0x030F	Buck/LDO15 Target Voltage
...		
PD30_VSEL_TARGET	0x031E	Buck/LDO30 Target Voltage
RSVD	0x031F	Reserved

1.10.4 DVC_GROUP Configuration⁶

DVC Group is ONLY supported on Bucks, not on LDOs.

PD_{A,B,C,D} is the encoding that is used to assign to one of the Bucks{0 .. 15} based on SW configuration for that device.

The Host SW is supposed to check the capabilities register before accessing the DVC group registers in a slave device. If the Host SW generates a write access to the virtual address space for a PD DVC registers (0x400 – 0x41B, 0x440 – 0x45B) for a slave PD that does not exist in the “PD mapping table” device, the device will generate a “DVC Error” back to the Host. Reads to PD registers, which are not in the “PD mapping table”, generate 0s.

Reads to unimplemented registers (0x41C – 0x43F, 0x45C – 0x4FF) will generate 0s, writes will be dropped, no error is generated.

To enable access to multiple Bucks/LDOs (aka PD) with a single command, there are two sets of registers that the host S/W will program.

1. Configuration Registers: These are the registers that tell the slave, which of the PD blocks are in a particular group. Each group config uses 4 registers and contains the PD ID (0 – Buck0, etc) and a VALID bit per register. This allows a maximum of 4 PD in a group.
2. Group Target Registers: These are registers that will be written during a SPMI command to let the slave know which voltage each of the PD in the group has to be transitioned to.

The registers are shown below.

Table 1.21. DVC Group Configuration

Register	Address	[7]	[6:5]	[4:0]
DVC_GROUP_CFG0_0	0x0400	VALID	RESERVED	PD_A
DVC_GROUP_CFG0_1	0x0401	VALID	RESERVED	PD_B
DVC_GROUP_CFG0_2	0x0402	VALID	RESERVED	PD_C
DVC_GROUP_CFG0_3	0x0403	VALID	RESERVED	PD_D
DVC_GROUP_CFG1_0	0x0404	VALID	RESERVED	PD_A

⁶DVC group capable PMU Specific.

Register	Address	[7]	[6:5]	[4:0]
DVC_GROUP_CFG1_1	0x0405	VALID	RESERVED	PD_B
DVC_GROUP_CFG1_2	0x0406	VALID	RESERVED	PD_C
DVC_GROUP_CFG1_3	0x0407	VALID	RESERVED	PD_D
...	...			
DVC_GROUP_CFG6_0	0x0418	VALID	RESERVED	PD_A
DVC_GROUP_CFG6_1	0x0419	VALID	RESERVED	PD_B
DVC_GROUP_CFG6_2	0x041A	VALID	RESERVED	PD_C
DVC_GROUP_CFG6_3	0x041B	VALID	RESERVED	PD_D

The CFG registers are typically based on the system power topology, so are unchanged during system operation. PD_A/PD_B/PD_C/PD_D are the ID for any of the 31 PD blocks that are assigned by S/W in that group. If the valid bit is not set, the lower bits are ignored.

Group configuration with non-contiguous VALID bit set is not supported.

The next set of registers are the Voltage target registers for each of the groups

Table 1.22. DVC Group Target Registers

Register	Address	Data [7:0]
DVC_GROUP_0_VSEL_TARGETA	0x0440	Target Voltage for PD_A in Group 0
DVC_GROUP_0_VSEL_TARGETB	0x0441	Target Voltage for PD_B in Group 0
DVC_GROUP_0_VSEL_TARGETC	0x0442	Target Voltage for PD_C in Group 0
DVC_GROUP_0_VSEL_TARGETD	0x0443	Target Voltage for PD_D in Group 0
DVC_GROUP_1_VSEL_TARGETA	0x0444	Target Voltage for PD_A in Group 1
DVC_GROUP_1_VSEL_TARGETB	0x0445	Target Voltage for PD_B in Group 1
DVC_GROUP_1_VSEL_TARGETC	0x0446	Target Voltage for PD_C in Group 1
DVC_GROUP_1_VSEL_TARGETD	0x0447	Target Voltage for PD_D in Group 1
...		
DVC_GROUP_6_VSEL_TARGETA	0x0458	Target Voltage for PD_A in Group 6
DVC_GROUP_6_VSEL_TARGETB	0x0459	Target Voltage for PD_B in Group 6
DVC_GROUP_6_VSEL_TARGETC	0x045A	Target Voltage for PD_C in Group 6
DVC_GROUP_6_VSEL_TARGETD	0x045B	Target Voltage for PD_D in Group 6

The target registers are assigned sequentially so that in one SPMI command, the host can send 4 bytes of data and each will be assigned to the appropriate PD in the group.

The DVC transitions for the Buck group have the following rules

1. If the valid bit is not set, the lower bits are ignored. It is possible for only one valid register in a Group, it behaves like a single DVC command.
2. If the Slew rate for DVC of all the valid PD in a group is the same, the transition will be sync-based transition (details in the next section).

3. If all the PD in a group don't have the same slew rate, the slave will do a time-based transition.
4. Done interrupt is sent after all the PD have finished transitioning to the target.

Command is considered good when the # bytes have been sent.

Invalid bytes are not supported. i.e.; if only 2 bucks are valid in a group and master sends a 1 byte or a 3 byte data command, its an Error.

Only one group will be addressed in one command.

1.11 Examples of SPMI transactions⁷

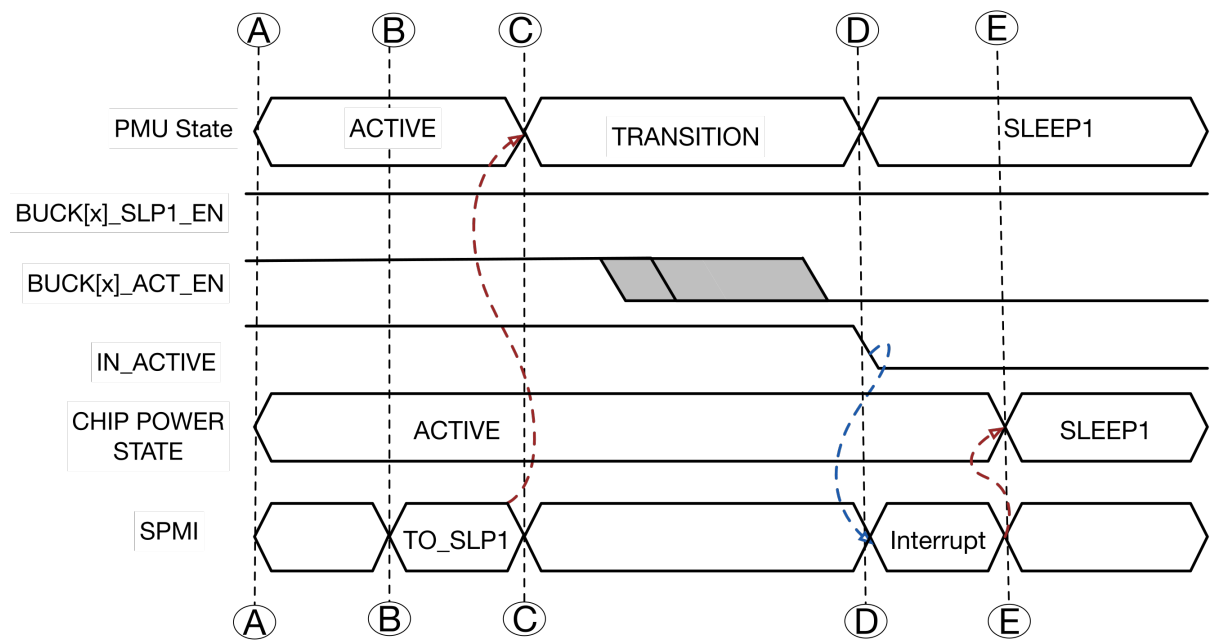
This section shows a few sequences of transactions on SPMI.

1.11.1 Power State Transition

Configuration:

- Power_State0 = ACTIVE
- Power_State1 = SLEEP1
- ACTIVE_TO_SLP1 is a valid transition in the Power sequence FSM
- Force_State register is not set.

Figure 1.12. System Power State transition



⁷System PMU Specific

- A. System is in ACTIVE state
- B. Host does a register write that sets "TO_SLEEP1" bit in the SPS register
- C. PMU starts the transition from ACTIVE to SLEEP1. All the Bucks that have ACTIVE_EN=1 and SLP1_EN=0 are transitioned from ON to OFF
- D. All the rail transitions have completed. PMU generates a MASTER_WRITE (interrupt) transaction on SPMI bus.
- E. Based on the interrupt bit set, Host knows that SPS transition to Sleep1 has completed.

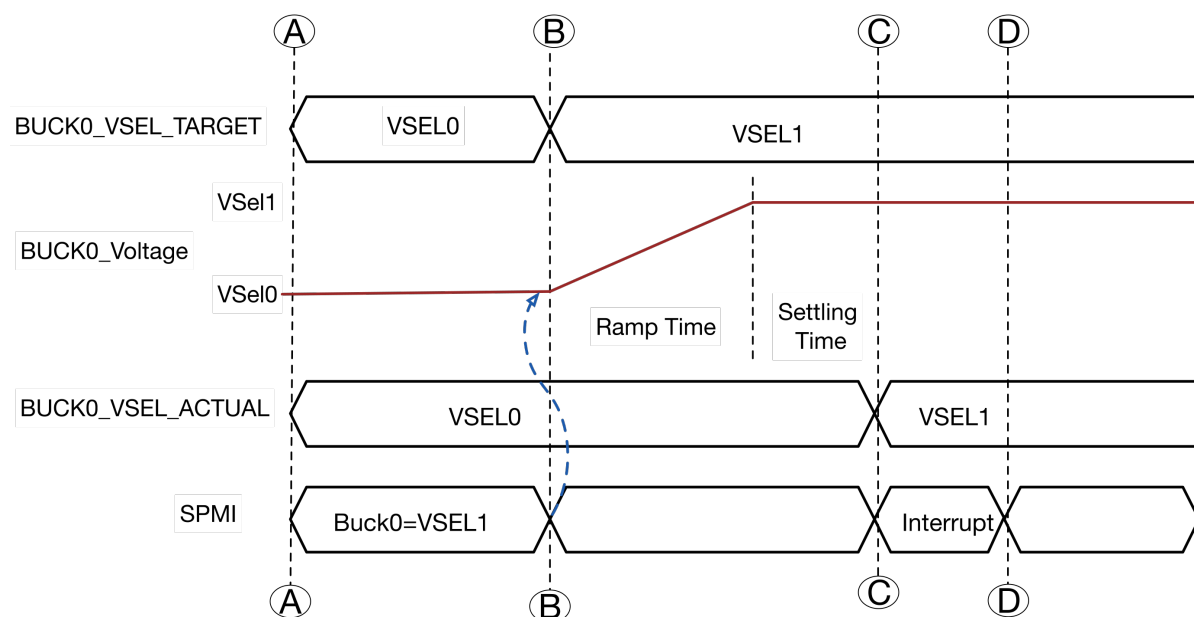
1.11.2 Single Rail DVC Transition⁸

Configuration:

- PD0 = Buck0
- Buck0_interrupt = 0x201
- Buck0_DVC_Enabled = 1

In this example, Buck0 is at a voltage VSEL0 and Master generates a register write to the VSEL_TARGET register with a value of VSEL1

Figure 1.13. Buck DVC transition



- A. Buck0 voltage is at VSEL0, master generates a register write to register Buck0_Vsel_target with a value of Vsel1. i.e.; Reg[0x300 = (VSEL1)]

⁸DVC capable PMU Specific

- B. PMU gets the new Vsel target, updates the vsel_target register and starts ramping the voltage from vsel0 to vsel1. In addition to the programmed ramp rate, there is also a settling time defined.
- C. Buck0 voltage has settled to Vsel1. PMU updates the Buck0_vsel_target register to VSEL1 and generates a master write (interrupt value = 0x201) transaction on SPMI to indicate to host that DVC for Buck0 is done.
- D. Based on the interrupt bit set, Host knows that DVC transition for Buck0 has completed.

1.11.3 DVC Group Transition⁹

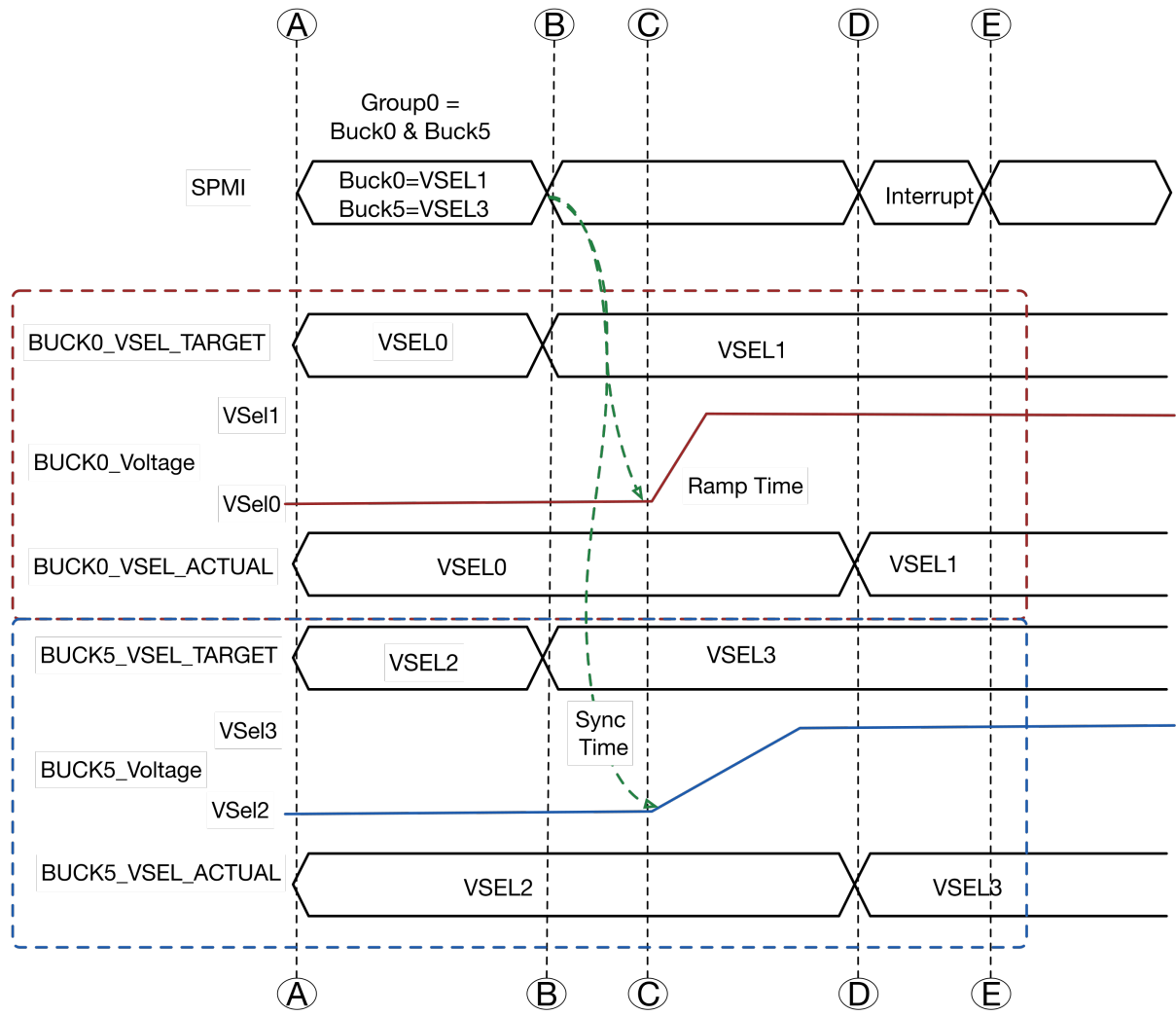
Configuration:

- DVC_GROUP0_CFG0 = '10000000 (Buck0 valid)
- DVC_GROUP0_CFG1 = '10000101 (Buck5 valid)
- DVC_GROUP0_CFG2 = '0000xxxx (invalid)
- DVC_GROUP0_CFG3 = '0000xxxx (invalid)
- Group0_interrupt = 0x193
- DVC2_OTP_En = 1
- Buck0_DVC_Enabled = 1
- Buck0_ramp_rate = X
- Buck5_DVC_Enabled = 1
- Buck5_ramp_rate = Y

The next transition diagram shows a group ramp that has different Ramp Rates specified for the Bucks in that group. In this case, each of the Bucks will follow their respective Ramp rates, but the start of the ramp will be aligned by the PMU.

⁹DVC group capable PMU Specific

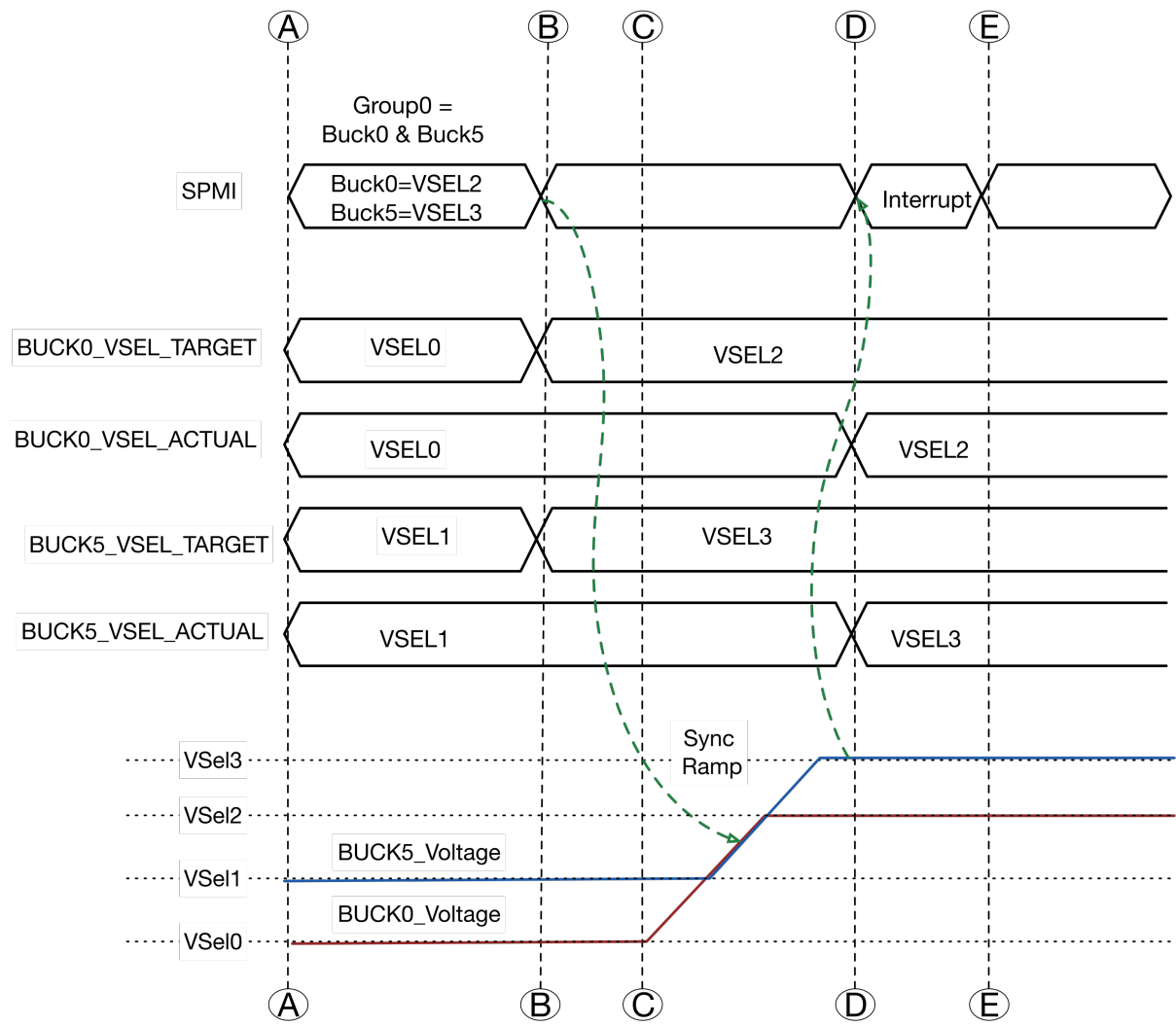
Figure 1.14. Buck group transition (Time Domain)



- Group0 has Buck0 and Buck5 as part of the group. Buck0 is at Vsel0 and Buck5 is at Vsel2. Host generates a register write transaction with 2 data bytes, Buck0 to vsel1 and Buck5 to vsel3
- PMU gets the DVC_Group register write and synchronizes the 2 bucks for transition. Since the ramp rates are not the same, they follow the Time Domain based sync and start the ramp together, however, each Buck uses its own ramp rate.
- Bucks are ready for voltage change and they start ramping to their respective target voltages.
- All the rail transitions have completed. PMU generates a MASTER_WRITE (interrupt) transaction on SPMI bus.
- Based on the interrupt bit set, Host knows that DVC_group0 transition has completed.

The next transition diagram shows a group ramp that has the same Ramp rate specified for all the Bucks in that group.

Figure 1.15. Buck group transition (Voltage Domain)

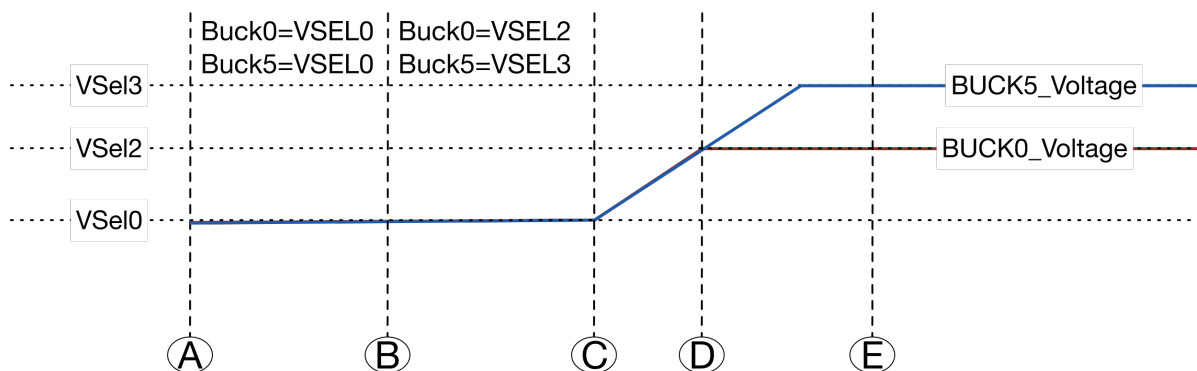


- Group0 has Buck0 and Buck5 as part of the group. Buck0 is at Vsel0 and Buck5 is at Vsel2. Host generates a register write transaction with 2 data bytes, Buck0 to vsel1 and Buck5 to vsel3
- PMU gets the DVC_Group register write and synchronizes the 2 bucks for transition. Since the ramp rates are the same, they follow the "coincidental" sync and start the ramp together. This is referred to as "Voltage domain" group DVC transition.
- Buck0 is ready for voltage change and starts ramping to the Buck5 voltage level. Buck5 is not ramping and waiting for Buck0 to get to the same voltage level. Once it gets there, both the Bucks ramp to their respective target voltages.
- All the rail transitions have completed, the voltages have settled. PMU generates a MASTER_WRITE (interrupt) transaction on SPMI bus.
- Based on the interrupt bit set, Host knows that DVC_group0 transition has completed.

The voltage levels shown in the earlier figures are meant to represent examples of the Buck transitions.

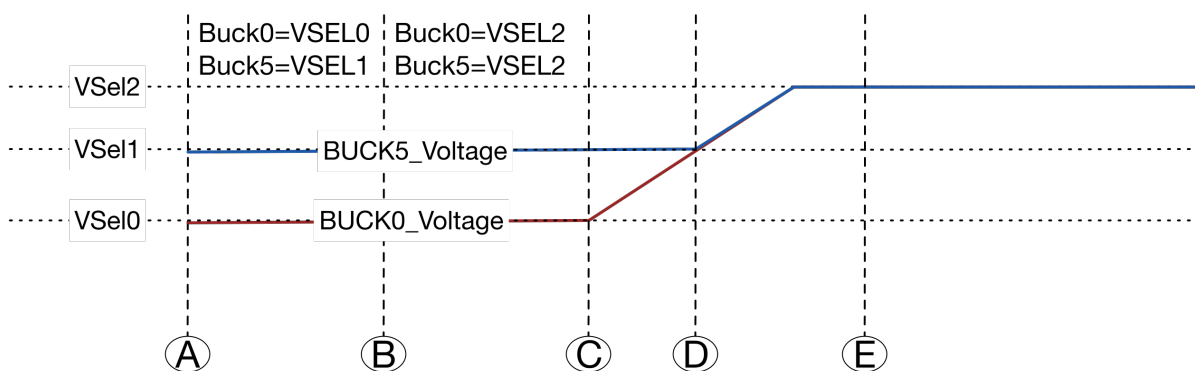
1. Same initial voltage (could be 0 or the same initial VSEL), but the target is different, this could either be different VSEL_TARGET values written by Host or due to VSEL_MAX value limiting the Buck voltage on one of the Bucks. In this scenario, the Bucks will ramp to the common portion together and then the higher one continues.

Figure 1.16. Buck group transition (different target)



2. Different starting VSEL, but the Target_VSEL is the same. Here the higher VSEL Buck will wait for the lower one to catch up and they will ramp together.

Figure 1.17. Buck group transition (same target)



1.11.4 DVC Early_Done¹⁰

This section goes over the DVC Early_Done specification.

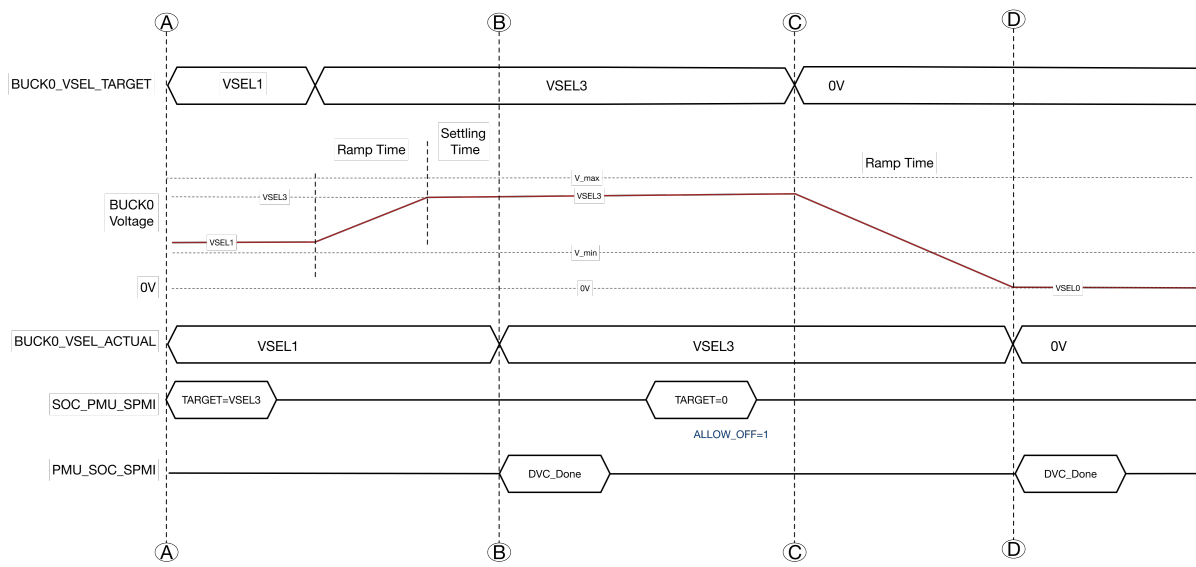
DVC_Early_Done is a requirement that will be specified in the ERS/configured in OTP for the particular slave design.

If required in the ERS and supported in the design, this section goes over the expected behavior of the "Early_Done" feature.

The timing diagram below shows the expected behavior for a Buck transitioning between various VSEL targets.

¹⁰DVC capable PMU Specific

Figure 1.18. Buck DVC (Allow_off=1)



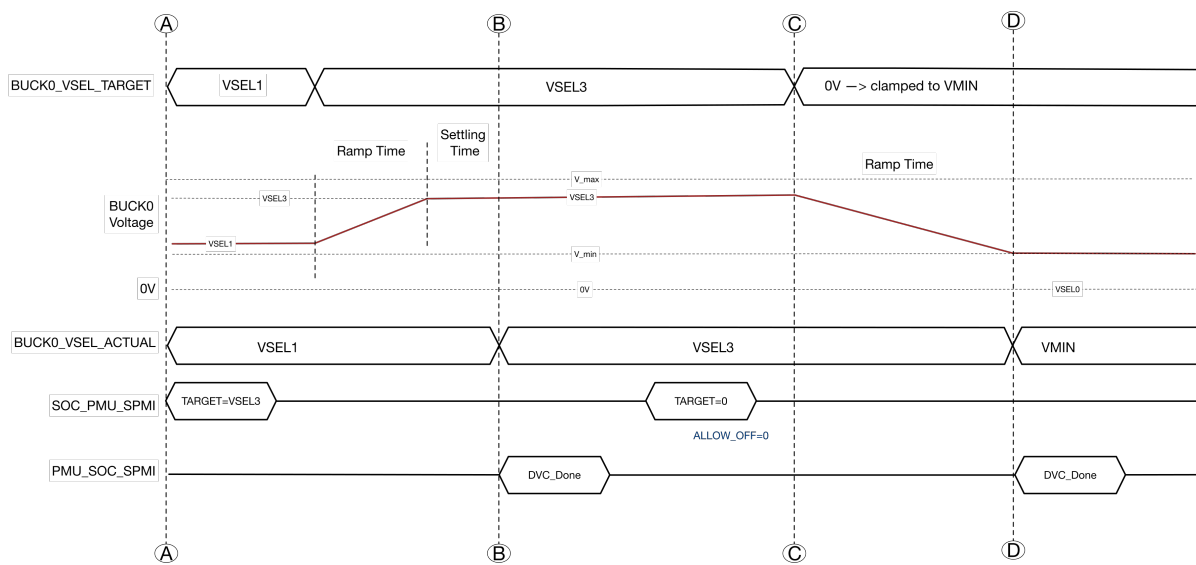
- A. Host generates a Target=VSEL3, the slave starts a transition
- B. After the Buck/LDO has finished transitioning to the new target (and settled at the target voltage), a DVC_Done interrupt is sent to the Host.

Master generates a new target=0 DVC command to the slave, since the configuration is "Allow_off=1" the expected target for the Buck is OFF (0V)

- C. Buck starts ramping down to 0V
- D. Once Buck has reached 0V, it sends a DVC_Done interrupt to the Host.

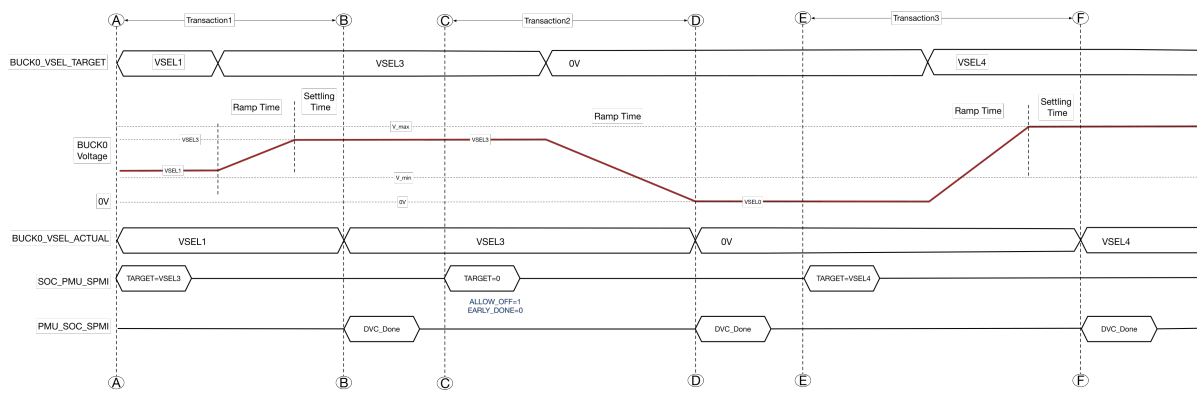
The figure below is the same as above, except that Allow_off=0 i.e.; the Buck will clamp to the VMIN voltage when Host generates a target_vsel=0 command on SPMI.

Figure 1.19. Buck DVC (Allow_off=0)



The figure below shows 3 transactions on a single Buck – the same behavior is expected for a DVC-group.

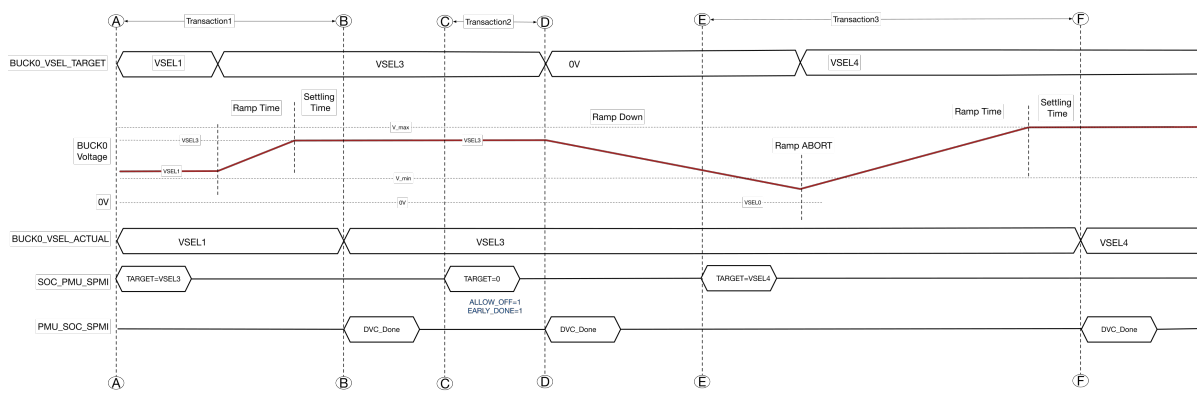
Figure 1.20. Buck DVC (Allow_off=1, Early_Done=0)



- A. Transaction 1: Host generates a Target=VSEL3, the slave starts a transition
- B. After the Buck/LDO has finished transitioning to the new target (and settled at the target voltage), a DVC_Done interrupt is sent to the Host.
- C. Transaction 2: Host generates a new target=0 DVC command to the slave, since the configuration is "Allow_off=1, Early_Done=0" the expected target for the Buck is OFF (0V). Buck starts ramping down to 0V
- D. Once Buck has reached 0V, it sends a DVC_Done interrupt to the Host.
- E. Transaction 3: Host generates a new target=vsel4 to slave
- F. same as B, but for the new target (vsel4).

The next figure shows the behavior with "Early_Done=1" for the same 3 transactions as above.

Figure 1.21. Buck DVC (Allow_off=1, Early_Done=1)



The difference in the flow is that Slave generates a DVC_Done message (D) as soon as it starts transitioning to the lower vsel. This is an indication to the Master that the slave can receive a new DVC command.

When the new DVC command shows up (E), the slave needs to abort the transition to Vsel0 and start the ramp to vsel4 (new target) without generating a DVC Error.

1.11.5 Backlight Control¹¹

Current designs use DWI/PSPI for Backlight control.

There are two types of messages that are used in existing designs.

Table 1.23. WLED - PSPI

WLED	WLED_PWM[10:0]	Backlight PWM duty cycle		
	WLED_ISET[10:0]	Backlight current setting		
	Byte0	Byte1	Byte2	Byte3
WLED	0010_0000	WLED_PWM[7:0]	0,WLED_ISET[14:11], WLED_ISET[1:0],WLED_ISET[10]	WLED_ISET[9:2]

WLED command (0x20) has 2 sets of data that is sent to the WLED device. The PWM duty cycle WLED_PWM[10:0] for which only 8b are used, and the backlight current setting WLED_ISET[14:0]

Table 1.24. BLC - PSPI

BLC	CHANNEL[3:0]	Target backlight channel		
	LEVEL[15:0]	Brightness Level		
	Byte0	Byte1	Byte2	Byte3
BLC	0011_0000	0000,CHANNEL[3:0]	LEVEL[15:8]	LEVEL[7:0]

BLC command (0x30) has 2 sets of data that is sent to the WLED device. The 4b Channel target CHANNEL[3:0], and the brightness LEVEL[15:0]

These commands will be sent over SPMI using the ERWL (Extended Register Write Long) transaction type:

Table 1.25. WLED & BLC - SPMI

	Command Frame		Address		1-8 Bytes of Data			
ERWL	SA[3:0]	0x0 - 0x7	ADDR[15:8]	ADDR[7:0]	Data0[7:0]	Data1[7:0]	Data2[7:0]	Data3[7:0]
WLED	SA[3:0]	0000_0011	WLED_ADDR[15:8]	WLED_ADDR[7:0]	WLED_ISET[7:0]	WLED_ISET[15:8]		

¹¹Backlight capable device Specific

	Command Frame		Address		1-8 Bytes of Data			
WLED_	SA[3:0]	0000_	WLED_ADDR [15:8]	WLED_ADDR [7:0]	WLED_ISET [7:0]	WLED_ISET [15:8]	WLED_PWM [7:0]	WLED_PWM [15:8]
Legacy	0011	0011						
BLC	SA[3:0]	0000_	BLC_ADDR [15:8]	BLC_ADDR [7:0]	0000, CHANNEL [3:0]	LEVEL [7:0]	LEVEL [15:8]	
	0011	0010						

Today's design supports both ITR (immediate transfer) and STR (synchronized transfer) modes. On SPMI, the synchronization happens outside the SPMI engine.

There is a dedicated queue for backlight control and when the transaction is put in the Q, the engine will try to send it out on the bus as soon as it can (based on arbitration logic).

To support WLED and WLED_Legacy commands, there is RWT bit in the device that selects whether PWM updates are supported or not. The WLED & BLC data is LSB first and then MSB, unlike the address which is MSB first & then LSB.

1.12 ERROR Handling

From the spec

ACK/NACK consists of two cycles that are used to confirm correct completion of a Write Command Sequence. ACK/NACK is not used on Command Sequences where the correctness of the Command Sequence can be confirmed based upon Data Frames received by the Device generating the Command Sequence.

If a Master receives a Command Sequence with an incorrect parity value in an Address Frame, it shall ignore the Command Sequence.

If the Address Frame parity error occurs in a Write Command Sequence, the Master shall inform the RCS of the error using NACK.

There are three scenarios where a RCS needs to handle errors.

1. When the Host generates a NACK for any of the transactions.
2. When the RCS generates a NACK for any of the transactions.
3. When the Bus Idle time out expires for any of the transactions.

For each of the error conditions,

[System PMU] NACK_RESET_EN=1. Transition the MFSM to OFF when a NACK is received from the SPMI RCS.

[Other Devices] If NACK_RESET_EN is supported, NACK_RESET_EN=1. Reset the slave interface when a NACK is received from the SPMI RCS. If NACK_RESET_EN is not supported, do Error handling based on AUTO_DISABLE register setting for IRQH Enable.

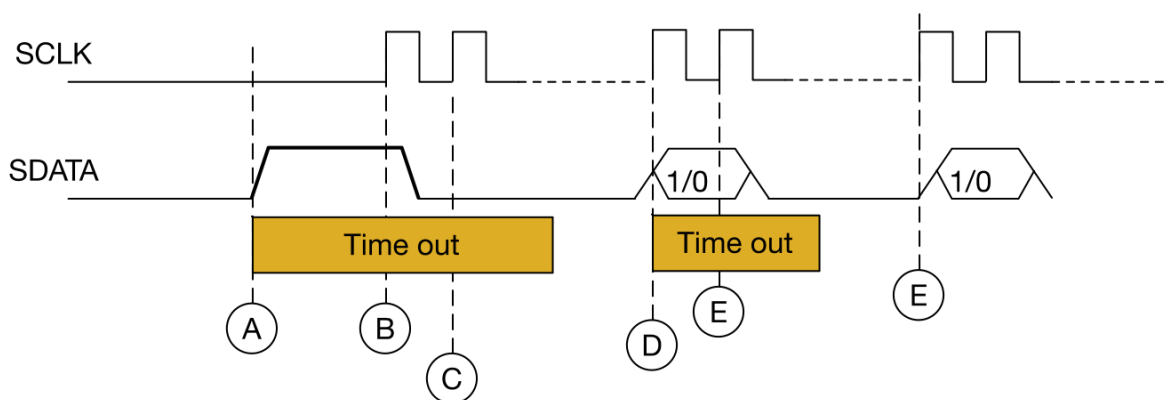
[All Devices] If AUTO_DISABLE=1, clear the RCS capability in the slave (IRQH enable = 0) and stop sending RCS commands. If AUTO_DISABLE=0, retry the transaction if IRQH enable is 1.

This section goes over the various error cases and shows examples of the expected behavior.

1.12.1 RCS – Master Write

The figure below shows the behavior for a RCS generated Master Write transaction.

Figure 1.22. RCS initiated Master Write transaction



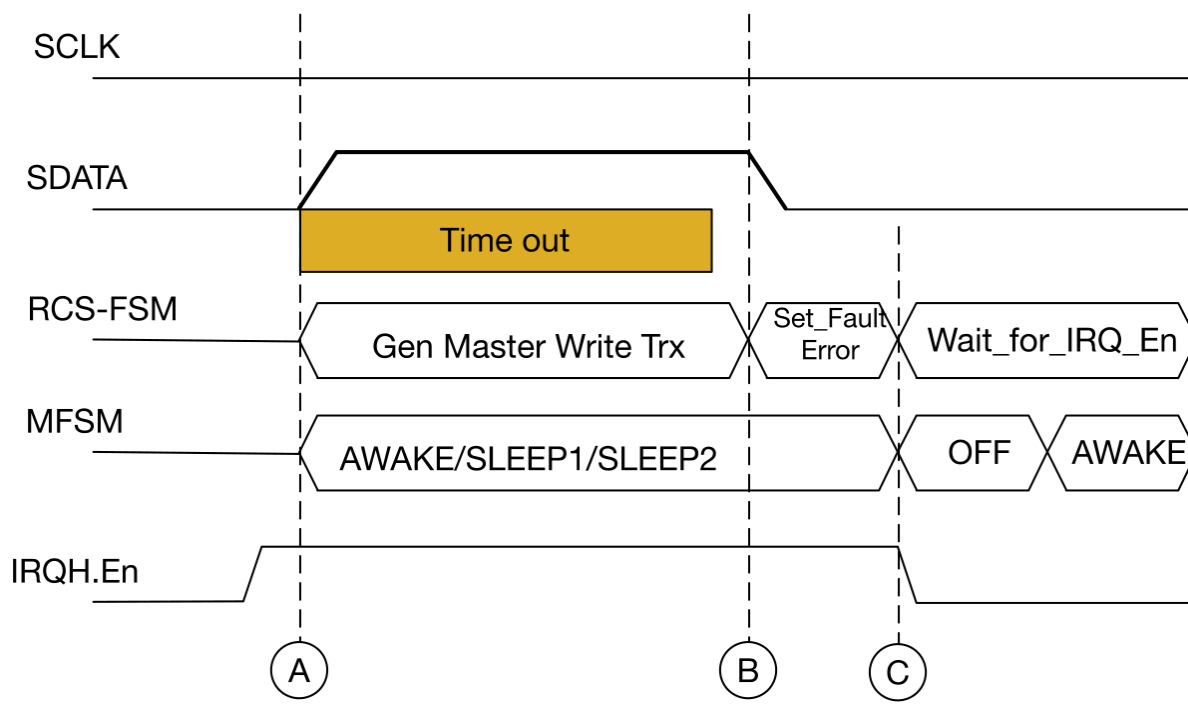
This is the sequence of events ..

- A. RCS has a pending transaction and pulls SDATA high
- B. RCS samples SCLK generated by the Host before the Timer expires, it stops driving SDATA
- C. SDATA is de-asserted by RCS
- D. The command packet is generated by RCS
- E. Address, Data ... etc.

1.12.2 RCS Transaction Start Error

In the first error scenario, after RCS has asserted SDATA, Host does not generate SCLK, as shown in the figure below.

Figure 1.23. RCS, Master Write – Start Error



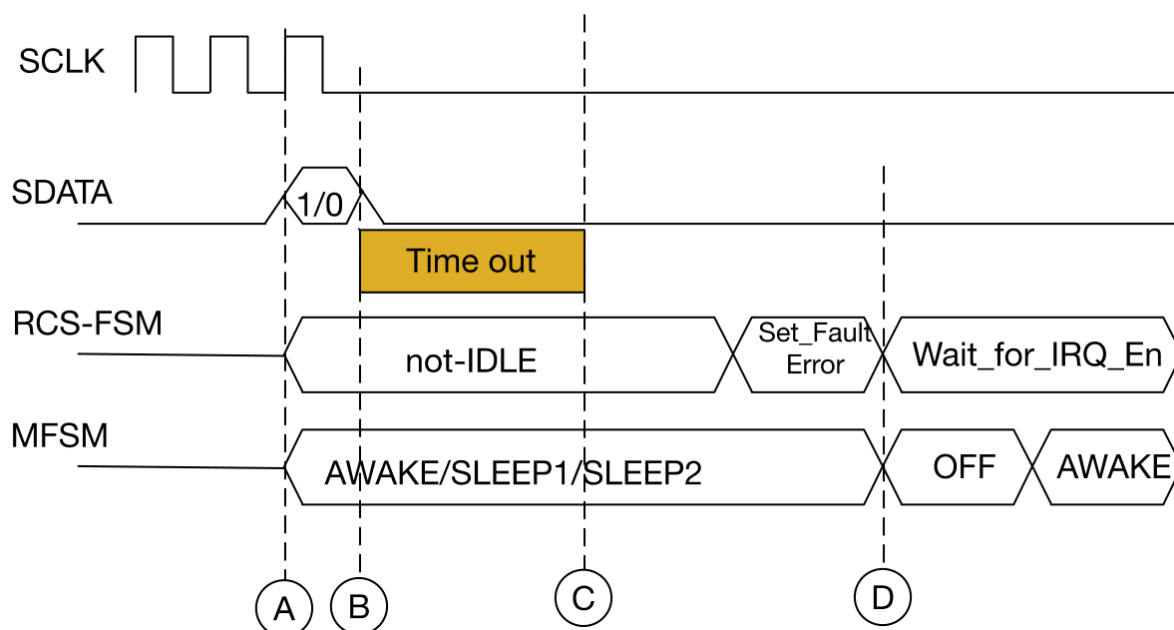
This is the sequence of events ..

- A. RCS has a pending transaction and pulls SDATA high
- B. Timeout timer expires, RCS has not sampled SCLK , RCS sets the Fault Error condition
- C. RCS signals to MFSM to transition to OFF, during OFF the IRQ.En bit is cleared and RCS FSM moves to “wait for IRQ Enable” state.

1.12.3 SPMI Protocol Error

In the second error scenario, the RCS FSM has started tracking a transaction on the bus, but there is a protocol error when the host does not generate SCLK within the timeout period, as shown in the figure below.

Figure 1.24. RCS, Master Write – Protocol Error



This is the sequence of events ..

- A. A transaction is ongoing on the SPMI bus
- B. Timeout timer set
- C. Timeout timer expires, RCS has not sampled SCLK , RCS sets the Fault Error condition
- D. RCS signals to MFSM to transition to OFF

1.13 Additional Notes

DVC2 vs. DVCu

- If the ramp rate is the same for all bucks in the group, the behavior is DVCu (voltage domain).
- If the ramp rate is different for **any** of the valid bucks in the group, the behavior is DVCu using the slowest ramp rate of the valid PD in the group.
- If the OTP bit is set, DVCu behavior is replaced with DVC2 (time domain).

DVC interrupt

- Interrupt is sent from the PMU after last buck finishes.
- Buck_vsel_actual will change as soon as the buck gets to the target value, it does not depend on the group.
- Host will not send a new DVC command for the same group before receiving a "DVC_Done" interrupt from the slave. If that happens, it's an Error and slave should not act on the second DVC command.

DVC Error

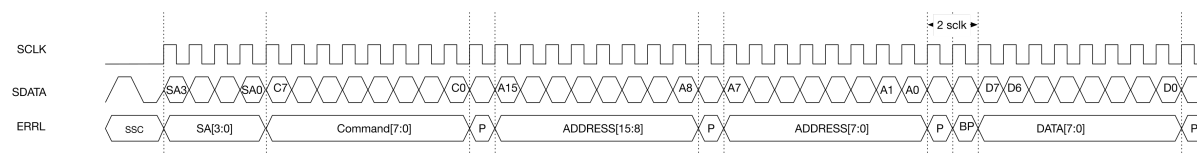
- If the error case (as defined earlier) happens, the PMU is expected to not use the new DVC target and send the host an error message that a new DVC request came before it signaled DVC_done.

Read Access from Master

- Each slave will have a specified set of registers that can be read without any inserted delay
- These are the “SPMI configuration” registers in the spec.
- Each slave will have a value programed in its “READ_LATENCY” register in the SPMI configuration that the master will read.
- Slave can expect that all reads to registers will have at least this minimum latency value.
- The Master will insert the latency delay in the “Bus Park” cycle of the read transaction.
- The delay is inserted by the master flat-lining the SPMI clock during this time.
- Slave should not be using the SPMI clock for internal access

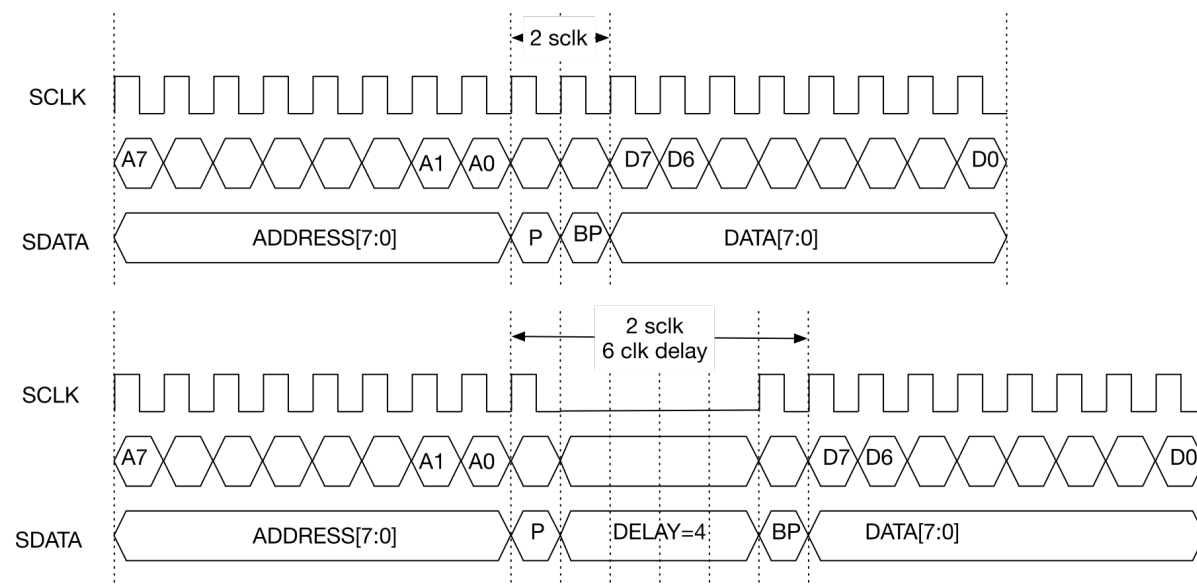
Extended Register Read Long

Figure 1.25. ERRL



an example with 4 clock delay

Figure 1.26. ERRL with Delay



Clearing Masked Events

When clearing masked event bits, the latch for the group interrupt is also cleared.

The usage model is unlikely to clear a single masked event bit and care about the result, and also because a spurious re-send of an interrupt is not a big deal if the interrupt is, in fact, still asserted.

Every time a slave goes through a slave reset, either through an explicit reset command or through a system Power_State=OFF power sequence, all the DVC_MASK registers are cleared & restored (if applicable) to their OTP values.

DVC Errors

The slave will implement separate error/event bits that distinguish different errors during DVC transitions:

- Out-of-sync errors: Example is a new DVC request from Master before the slave sends a 'DVC_done' transaction for that PD.
- Protocol errors: Example is incorrect programming of the PD (DVC-capable vs. not) or of the registers for the DVC group (Valid bits are not contiguous).

Power State Done Events

Special handling of Power_State_Done interrupts from the slave are in the cases when there is a crash sequence during a power state transition. In that case, it's okay for the slave to signal multiple "power_state_done" bits set to the Master.

During a Awake → Off → Awake transition, since the master does not expect "done" interrupt for the Off state, when the slave gets to Awake state, both Off and Awake bits are set; this will tell the Master that the slave did a off → awake transition.

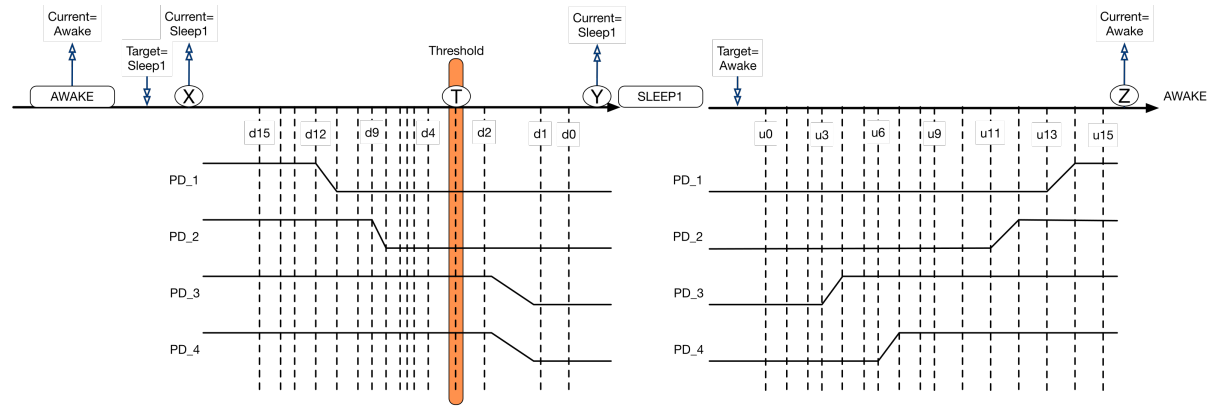
Slave Master Write transaction

- On entry to OFF (or CRASH), IRQH enable is set to 0
- PMU will not generate IRQH commands on OFF → ACTIVE transition until Host reads the "last interrupt" register, which sets the IRQH enable bit to 1.

1.13.1 Power State Transition [Examples]

This section will go over the Power State transitions in the Slave, with the focus on handling abort scenarios.

Figure 1.27. Power State transition (Awake → Sleep1)



An example being shown above using 4 PD (Bucks, LDOs, groups) that use some of the slots to transition between two Power states, AWAKE and SLEEP1, which support transition ABORT in the mFSM.

There are 2 “static” configurations in PMU, based on system design.

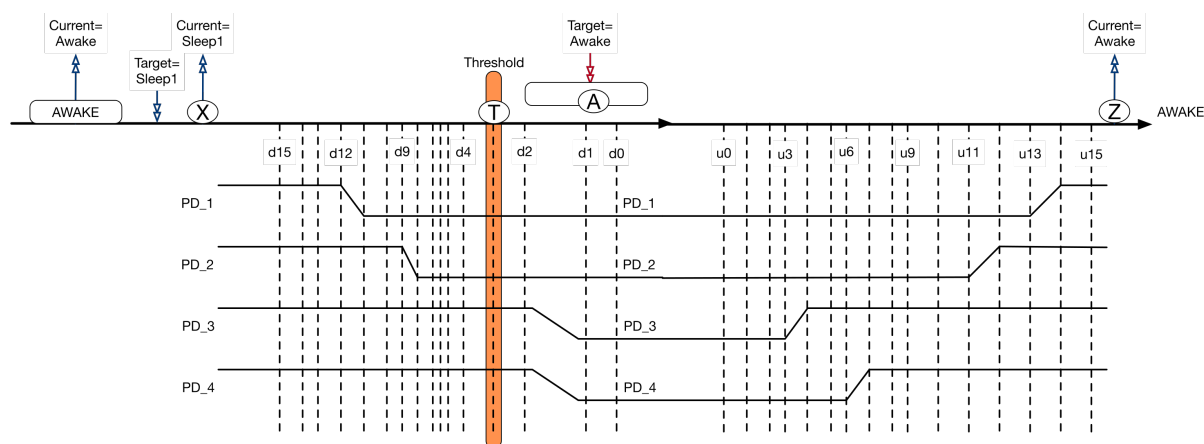
Abort support:

- Full Abort support. i.e.; generating the “Done” interrupt as soon as it gets the Target=Sleep1 command (point X in the figure above). This means that when the PMU generates the “Done”, it is ready for a new request from the Host.
- No Abort support. i.e.; generating the “Done” interrupt after the sequencer has finished the transition to Sleep1 (point Y in the figure above). This is legacy mode, where the “Done” is generated only after the power sequencer has finished the transition.

Threshold support:

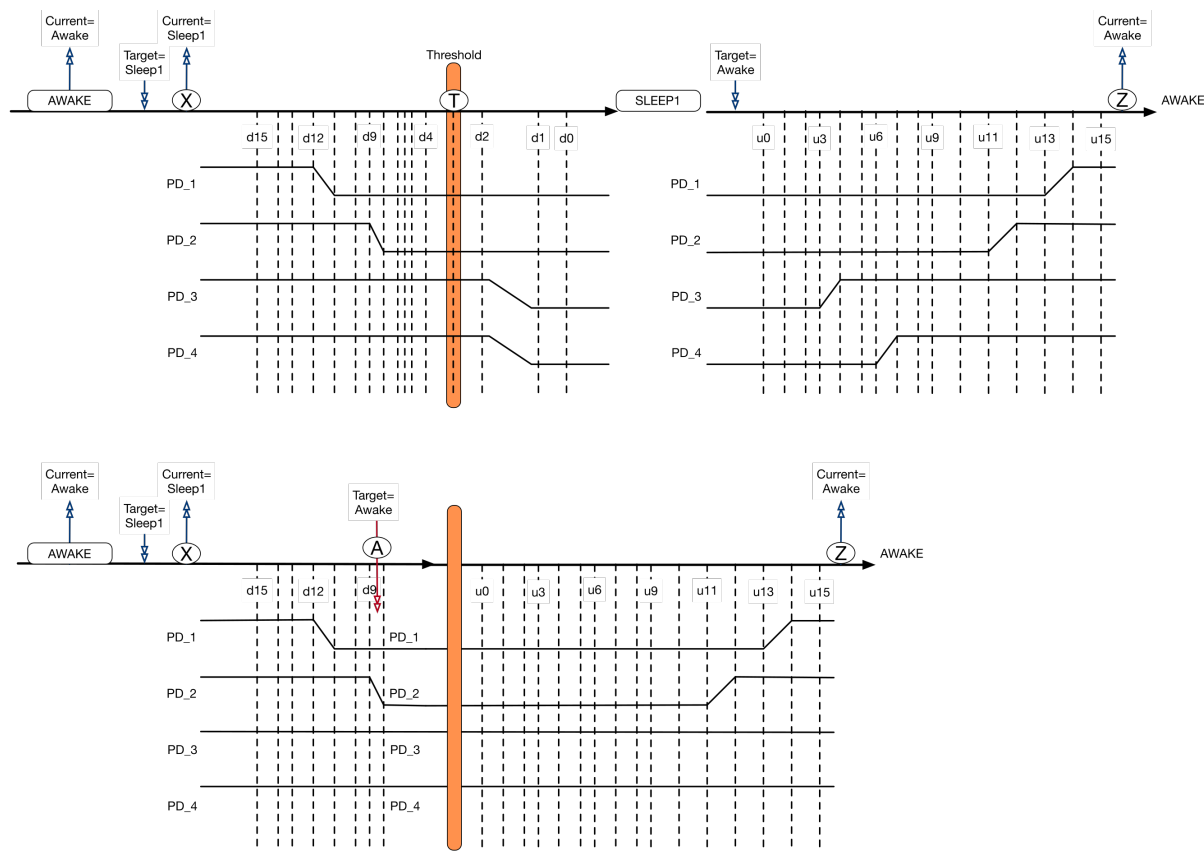
- Since multiple rails can be sequences in any slot and the ability to abort a “in-transition” rail can cause system power delivery issues, the PMU state machine supports a threshold (slot number) for the system to configure.
- If Abort is supported, the PMU will only honor it if the wake event happens before threshold slot. (slot d2, point T in figure above).
- Up transitions will always indicate “Done” only after sequencer has finished all transitions. (point Z in the figure above).

Figure 1.28. Power State transition (New Target after Threshold)



- If the PMU is configured to support Abort, it will signal Current=sleep1 immediately after getting Target=sleep1 command. (point X)
- If the wake event i.e. Target=Awake (point A in the above figure), shows up after the threshold, it behaves like there is no abort.
- The sequencer does the full down/up action is taken and interrupt is generated after all rails have transitioned. (point Z)

Figure 1.29. Power State transition (Awake → Sleep1 w/ Abort)



- If PMU is configured for abort support (current=sleep1 at point X).
- If the wake event i.e. Target=Awake (point A) shows up before the threshold (T), the down sequence is aborted (at the end of the current slot).
- PMU will start the full “UP” sequence; the rails that have not started transitions will not be affected.
- There is no assumption that the power down/up sequences are symmetric !

1.14 Non-compliance with MIPI Spec

MIPI Specification v2.0 Page 61, line 1206

10.3 Slave Communication Request on Uninitialized Bus

- Is not supported
- Slave communication request is only supported on Initialized Bus

1.15 Clarifications from earlier Specs

This section goes over the clarifications/errata and learning from systems that have earlier revisions of the spec (0.87)

1.15.1 Slave Hold Behavior

MIPI SPMI Spec:

Page 43, Line 900, Slave Request Hold

If a Request Capable Slave device requests and wins Slave Arbitration using the A/SR-bit, it shall not request Slave Arbitration again using the A/SR-bit until it has observed at least one arbitration process where the A/SR-bit is logic level zero.

Slave should exercise an empty arbitration wait (aka slave hold) only after sending its first request, not before sending the first one. If a slave has both A-bit & SR-bit requests pending, it will send both before doing the empty arbitration cycle.

Even if the slave does not have a pending IRQ, it needs to do an empty arbitration to be able to send out the request when it shows up in the queue. The timeout behavior in this condition (no pending IRQ) is the same as that with a pending IRQ.

If the slave is retrying the transaction due to a NACK or Error condition, it needs to follow the arbitration protocol and wait for an empty arbitration slot before doing the retry.

1.15.2 Command and Arbitration Timeout values

MIPI SPMI Spec:

Page 88, Line 1628, SPMI Bus Timeout Specification

Typical value is 96us.

There are two separate timeouts in the slave design and they should be set to different values.

1. Arbitration wait timeout: This is the timeout value that starts when SCLK is inactive. It is used to determine if the SCLK input line has not activity for a programmable period of time. If it expires without SCLK activity, the slave can start a new arbitration request by setting SDATA high. It restarts when SCLK activity is detected.

2. Arbitration Request/Command timeout: If there is no activity in the SPMI clock and the SPMI slave FSM is not in the IDLE state, it will time-out and go to IDLE state after this timer expires and reset the slave interface.

They should be set to different values so that they don't collide with each other, in addition a separate event should be sent for each of these timeout events.

If a command timeout occurs when the slave is monitoring the bus, it will trigger a separate event in the slave.

1.15.2.1 ADC data read back enhancement

The accumulator registers are arranged per slot GPADC_SCH_1_ACCUM_SLOT<0 .. 15>_RES<1 .. 4>

To read ADC results, SW needs to do 3 SPMI reads to get the data

- Read 4 Bytes of GPADC_SCH_1_ACCUM_SLOTx_RES<1 .. 4>
- Read 3 Bytes of GPADC_SCH_1_ACCUM_COUNT<1 ..3>
- Restart measurements by clearing GPADC_SCH_1_LAST

The optimization in this revision of the spec is for devices is to allow the transaction as a single 8B or 16B SPMI read transaction from the SoC.

An example of this would look like this:

- SW reads 8 Bytes of data using one SPMI transaction
- GPADC_SCH_1_ACCUM_SLOTx_RES<1..4> [Accumulator data]
- SLOTx_COUNT<1..3> [# samples]
- SLOTx_RESTART [Restart Accum on read]

1.15.3 Slave to Slave transactions

This section clarifies the expected behavior of devices wrt DVC_CAPABILITY_TYPE[3] support i.e.; SLAVE_TO_SLAVE

This is a READ-ONLY bit and its value is set based on the device design and system requirements. This bit can only be set for devices that have RCS=1.

If the slave_to_slave capability is enabled, i.e.; [3] = 1, the device can

- generate transactions in addition to Master Writes
- receive transactions from both Master and Slaves on the bus

If the slave_to_slave capability is disabled, i.e.; [3] = 0, the device must

- not be capable of generating any other commands on the bus except for Master_Write (if RCS_En=1)
- not accept any transactions on the bus that are not generated by the Master

As with other SPMI related error conditions, a detection of violation of these rules will result in the device generating an SPMI error IRQ to the Host.

1.15.4 Multiple Instantiations of same silicon device

In the case that a particular device is instantiated multiple times in a system, it needs to have the ability to distinguish itself from other "same part number" devices. This requires each device to either allocate an extra pin for bootstrapping (i.e. for a unique SLAVE_ID and PD_mapping), or require different OTPs for every part.